CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

THIẾT KẾ GIÁM SÁT

TS. Đỗ Như Tài
Đại Học Sài Gòn
dntai@sgu.edu.vn

Microservices Resilience, Observability and Monitoring

Microservices Resilience and Fault Tolerance

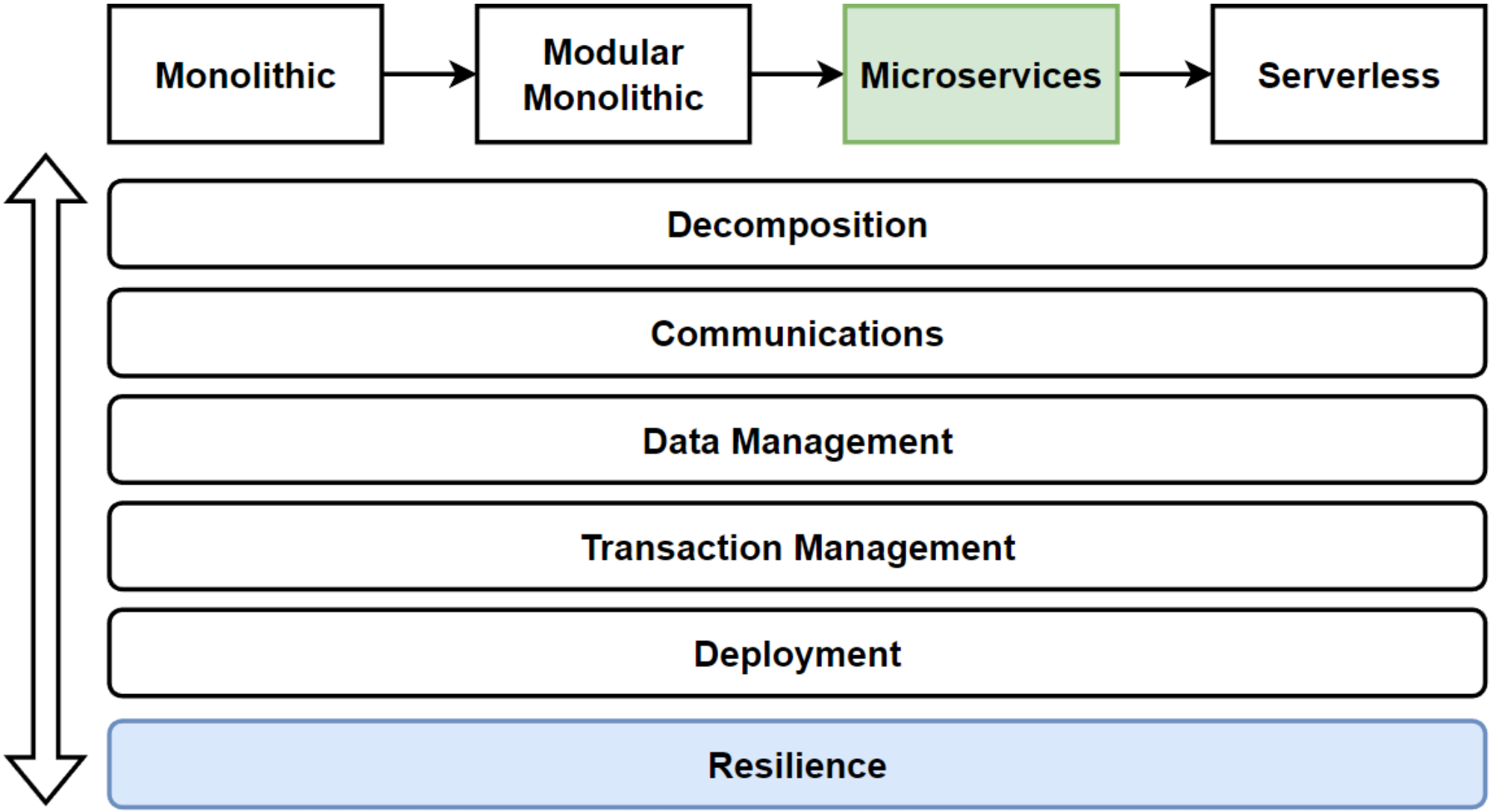
Retry, Circuit-Breaker, Bulkhead Pattern

Microservices Observability with Distributed Logging and Distributed Tracing

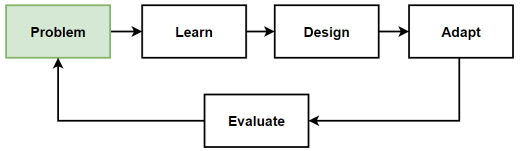
Elastic Stack which includes Elasticsearch + Logstash + Kibana

Microservices Health Monitoring

Architecture Design – Vertical Considerations



Problem: Fault tolerance Microservices able to remains operational for any failures or disruptions



Problems

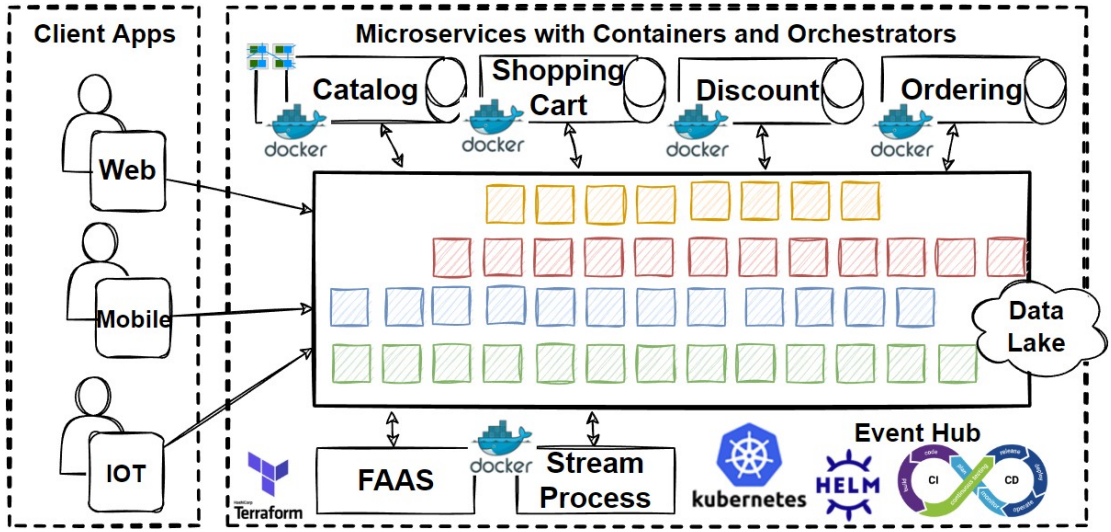
- Business teams able to deploy new features immediately to compete the market
- Due to heavy traffic, microservices got exceptions and broke some major use cases that cause lost revenue
- Create big crisis to customer loyalty

Considerations

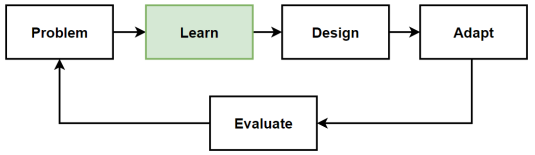
- Deploy new features immediately without affecting the system general behaviors
- System should recover from failures and provide ability of a system to withstands.
- Architecture should be designed to be resilient

Solutions

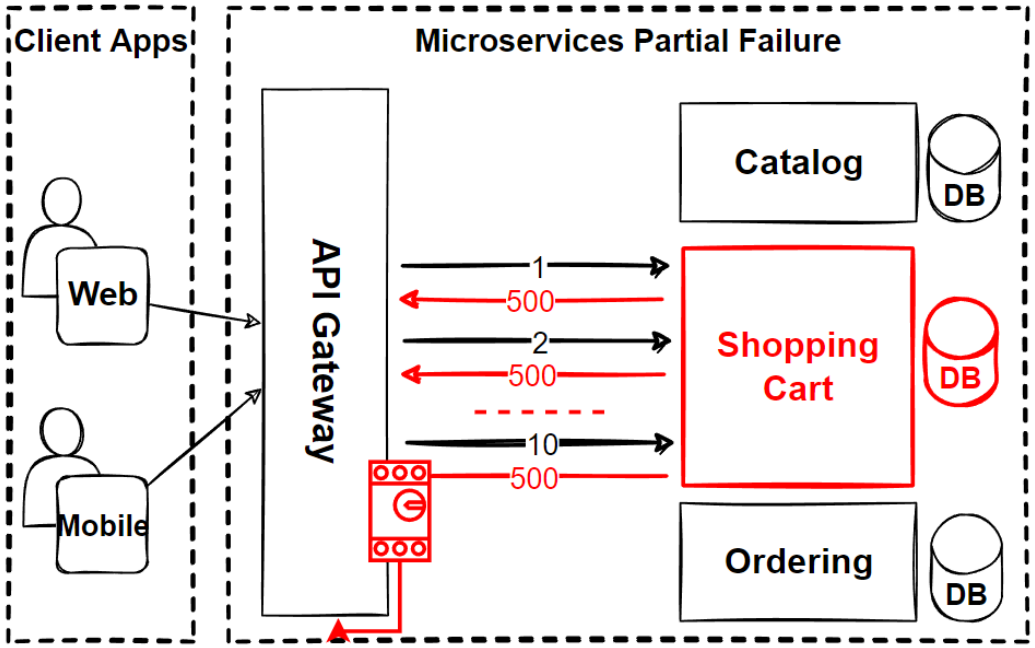
- Microservices Resilience and Fault Tolerance
- Microservices Observability with Distributed Logging and Distributed Tracing
- Microservices Health Monitoring



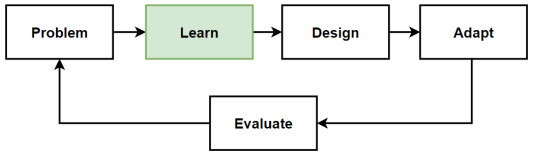
What is Microservices Resiliency ?



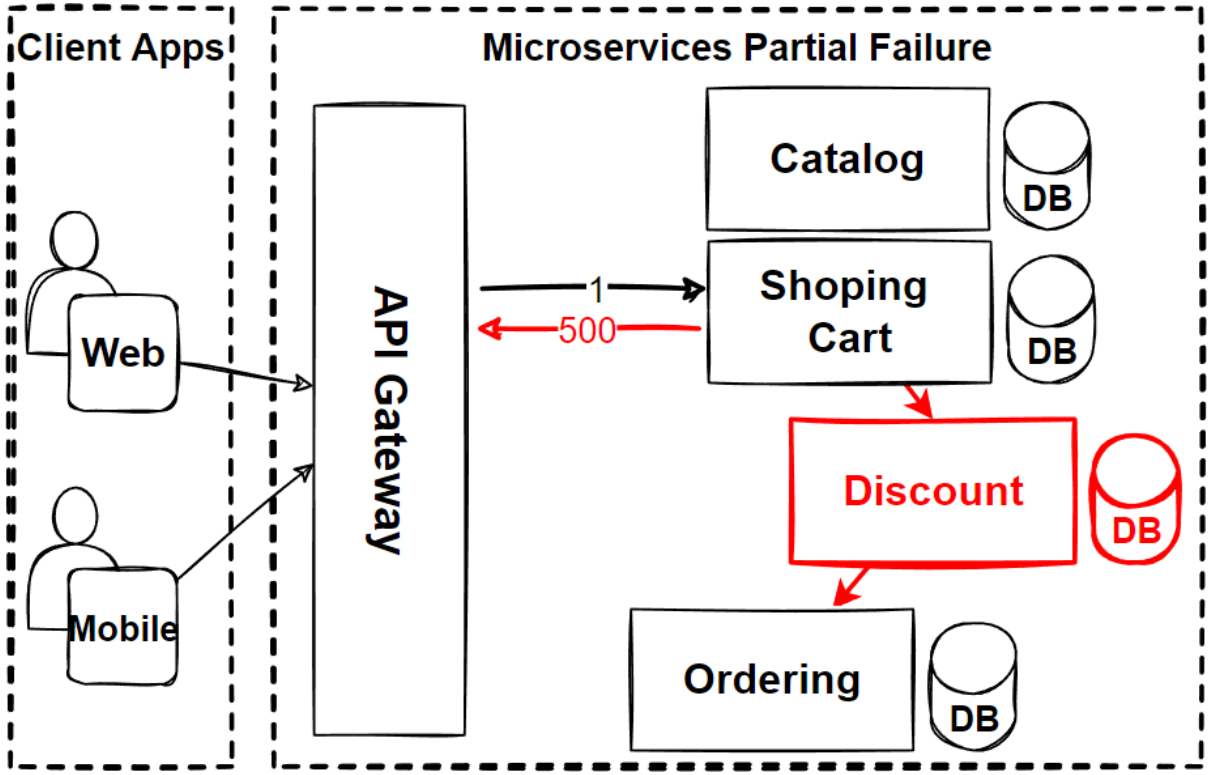
- **Microservice architecture** have become the **new model** for building modern **cloud-native applications**.
- But it is **increasing interactions between microservices** have created a **new set of problems**.
- Should assume that **failures will happen**, and **dealing** with **unexpected failures**, especially in a distributed system.
- **Network** or **container failures**, **microservices** must have a **strategy to retry** requests again.
- App is going to **fail at some point** and we need to **embrace failures**.
- **Design microservices with failures** that need to be prepared for the worst case scenario.
- **What happens when the machine where the microservice is running fails ?**



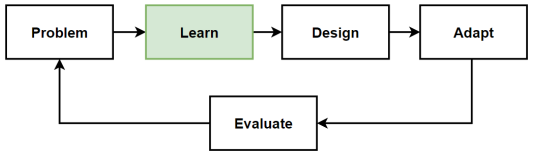
Cascade Failure in Microservices



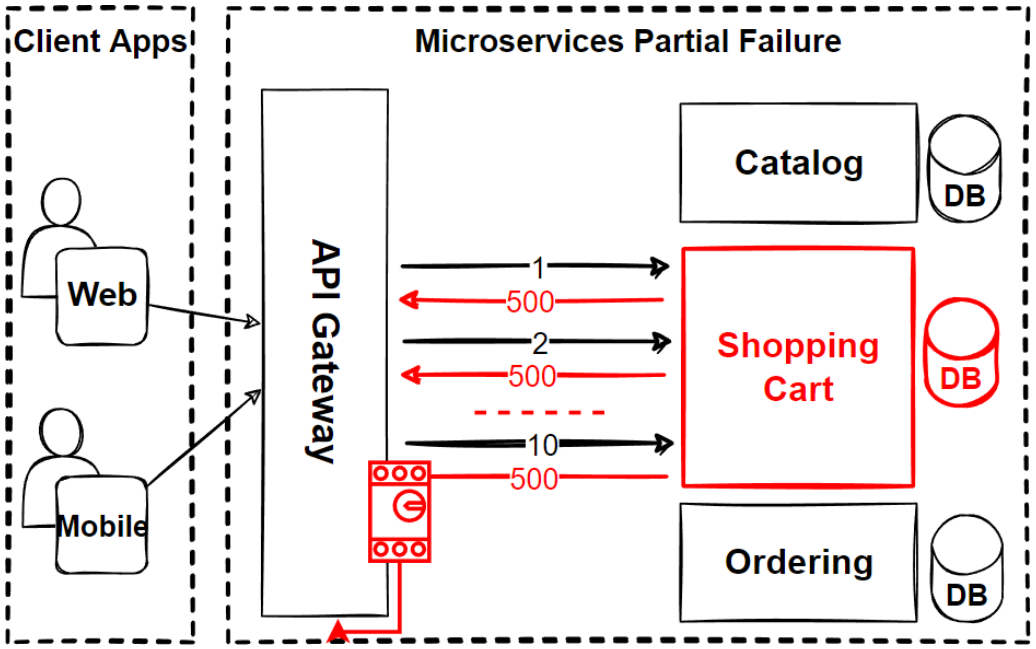
- **What happens** when the machine where the **microservice is running fails ?**
- **Single microservice can fail** or might not be available to respond for a short time.
- Since **clients** and **services** are **separate processes**, a service might **not be able to respond in time** for client's request.
- The **service might be overloaded** and **responding very slowly** to requests or might **not be accessible** for a short time because of **network issues**.
- **If Service A calls Service B,** which in turn calls Service C, **what happens when Service B is down ?**
- What is your **fallback plan** in such a scenario ?
- How do you **handle the complexity** that comes with **cloud** and **microservices ?**



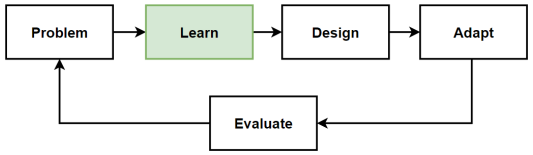
What is Microservices Resiliency ?



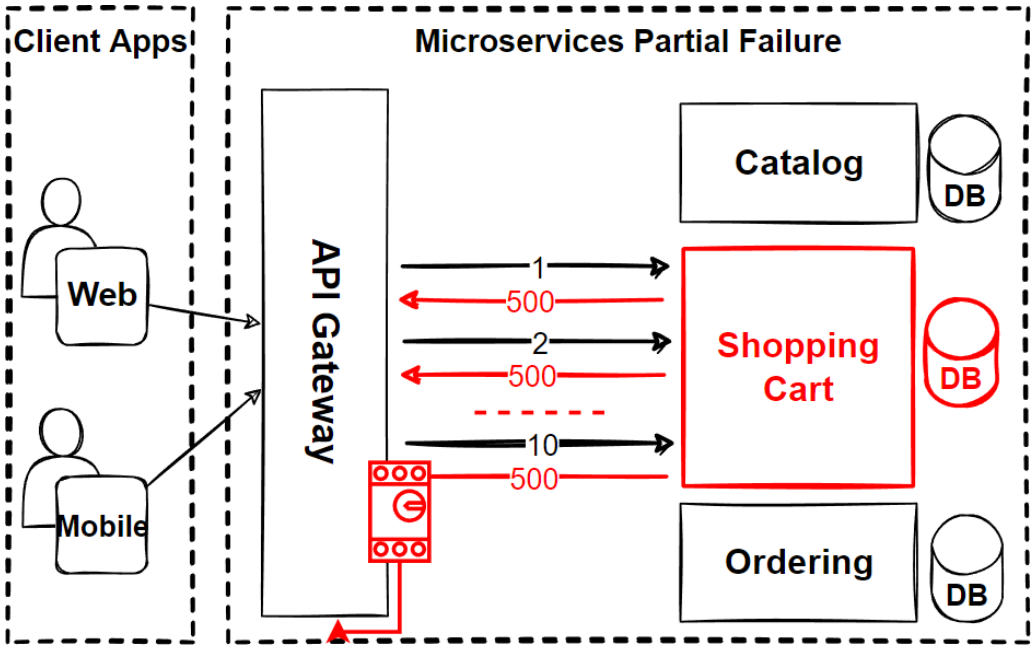
- **Microservice** should **design for resiliency**, needs to be **resilient to failures** and must **accept partial failures**.
- Design **microservices** to be **resilient for partial failures**, ability to **recover from failures** and **continue to function**.
- **Accepting failures** and responding to them without any downtime or data loss.
- **Return the application** to a **fully functioning state** after a failure.
- Assume that **failures will happen** and **design** our **microservices** for **resiliency**.
- **Microservices** are **going to fail** at some point, that's why we need to learn **embracing failures**.
- **Microservices** system is considered **to be resilient**, if it can continue to function effectively **despite failures or disruptions**.
- Should be **fault tolerant** and **handle failures gracefully**.



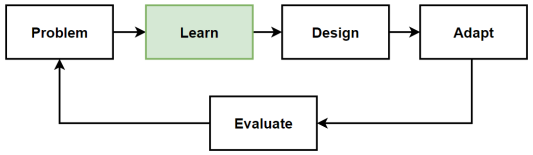
Microservices Resiliency Patterns



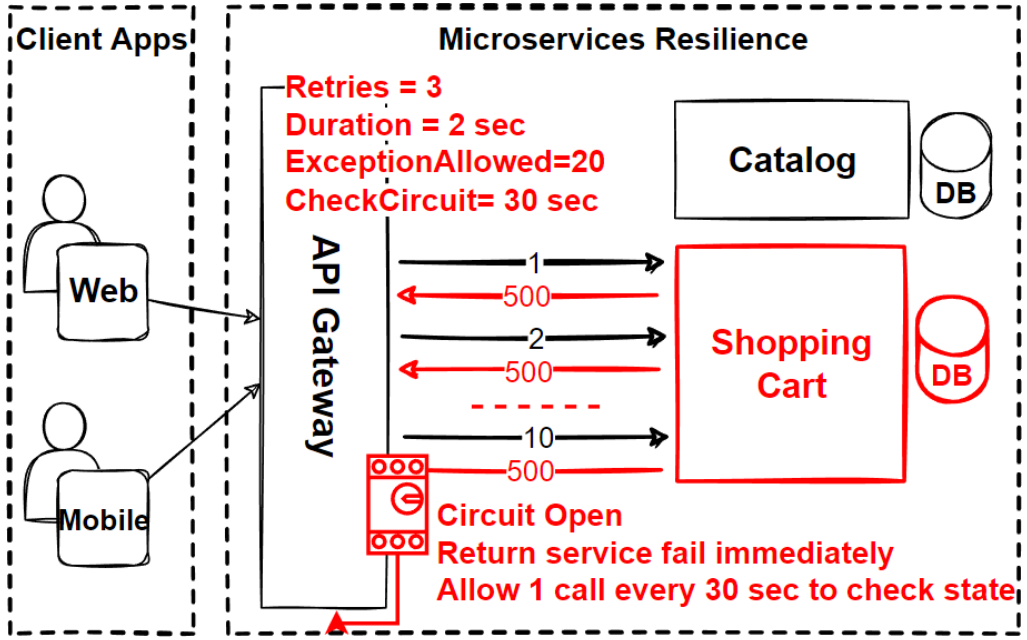
- To provide **unbroken microservice**, the **architecture** must be **designed correctly**.
- We can apply to provide **uninterrupted microservice**, with "**Resilience Patterns**".
- **Resilience Patterns** can be **divided into different categories** according to the problem area they solve.
- **Ensuring the durability** of services in microservice architecture is **relatively difficult** compared to a monolithic architecture.
- **The communication** between services is **distributed** and many **internal** or **external network traffic** is created for a transaction.
- Communication need between services and **dependence on external services increases**.
- The possibility of **occurring errors** will **increase**.
- There are **several patterns** that can be used to **improve** the **resiliency** of a **microservices system**.



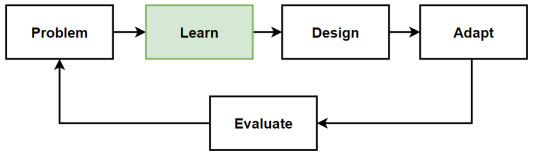
Microservices Resiliency Patterns - 2



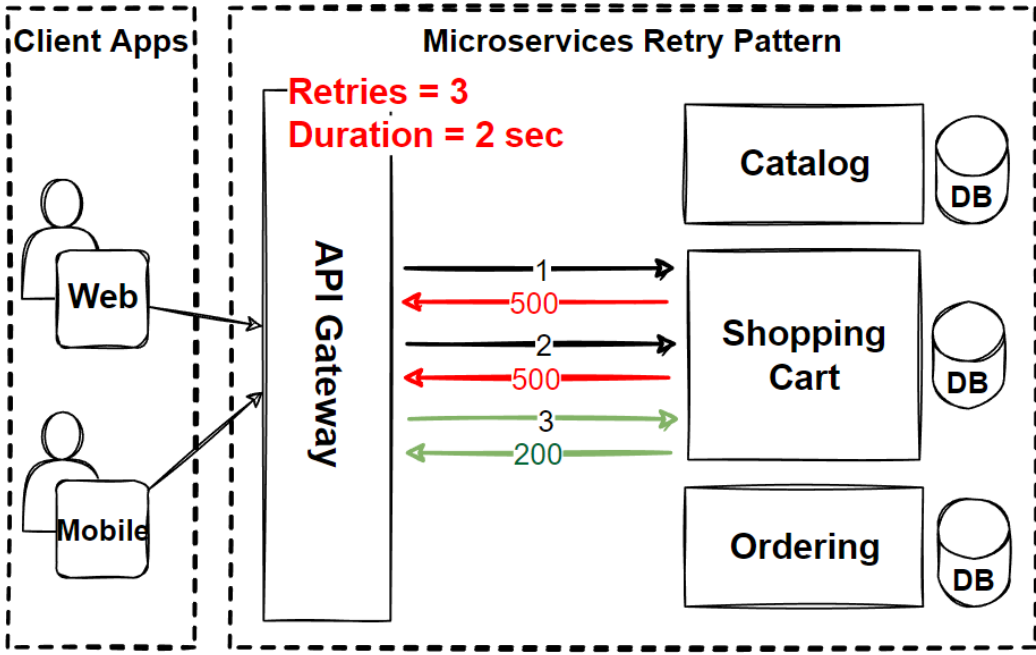
- **Retry Pattern**
  Retrying a request if it fails or times out. Retries can be implemented at the client or the service level, to handle temporary failures or disruptions.
- **Circuit Breaker Pattern**
  Introducing a proxy or "circuit breaker" between a client and a service. It will open the circuit and prevent further requests from being sent to the service.
- **Bulkhead Pattern**
  Partitioning a system into isolated components, or "bulkheads," to prevent the failure of one component from affecting the others.
- **Timeout Pattern**
  Should not wait for a service response for an indefinite amount of time, throw an exception instead of waiting too long.
- **Fallback Pattern**
  Providing an alternative behavior or response if a request fails or times out.



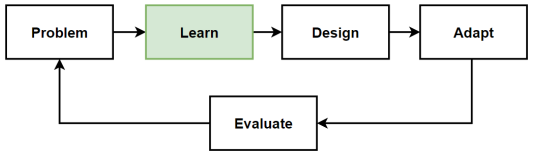
Retry Pattern



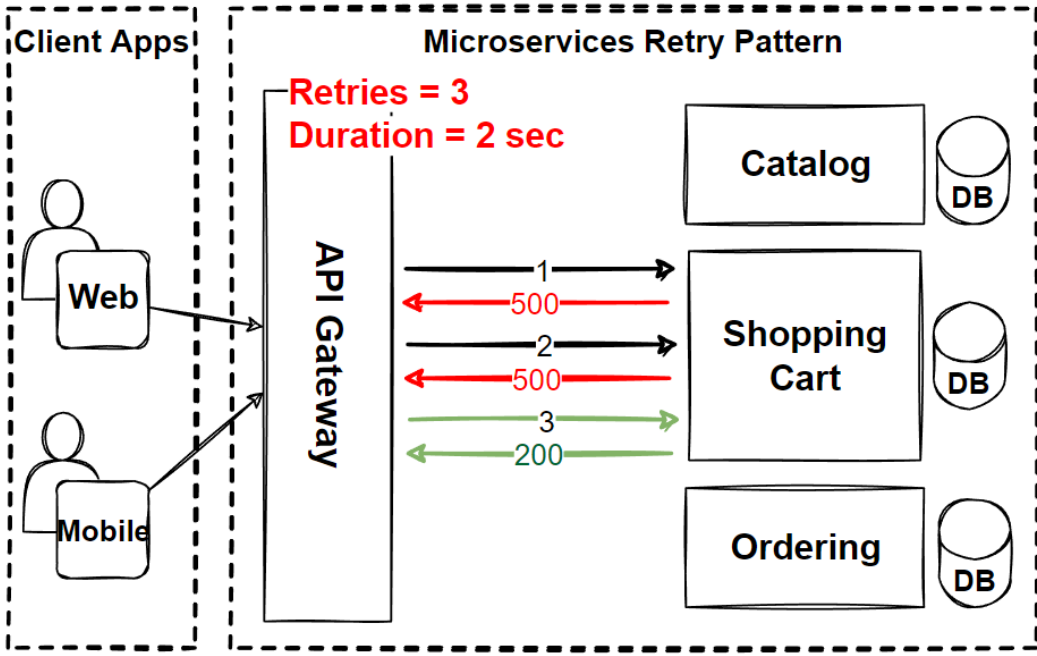
- **Inter-service communication** can be performed by **HTTP** or **gRPC** and can be managed by developments.
- When in comes to **network**, **server** and **such physical interruptions** may be **unavoidable**.
- If one of the services **returns HTTP 500** during the transaction, the **transaction interrupted** and an **error** may be returned.
- But **when** the user **restarts the same transaction**, it may work.
- **More logical** to **repeat the request** made to the **500 returned** service, user will be able to **perform the transaction successfully**.
- **Microservices communications** can **fail** because of **transient failures**, but this **failures** happen in a **short-time** and **can fix after** that **time**.
- For that cases, we should implement **Retry Pattern.**



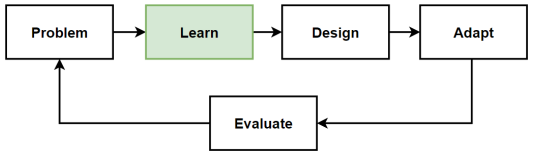
Retry Pattern - 2



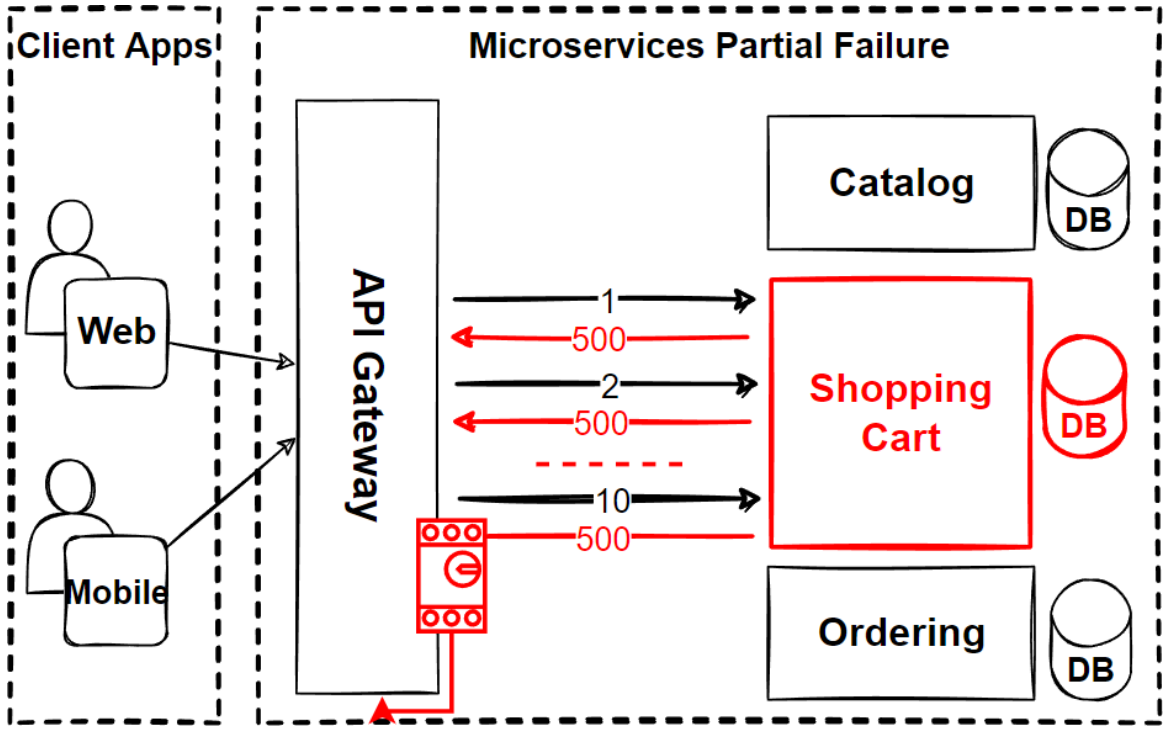
- **Retry Pattern** allows to **automatically retry** an operation **if it fails.**
- Ensure that a service is **able to handle failures** or **temporary unavailability** of dependent services.
- Common use of the R**etry pattern in microservices** is to handle **temporary failures** when **calling** a **downstream service**.
- I.e. **shopping cart** service that **relies on a payment service** to process payments.
- If the **payment** service is **temporarily unavailable** due to a **network issue**, the shopping cart service use the **retry pattern** to **automatically retry** the **payment request**.
- Allow the microservice time to **fix itself** with **self-correct**, and **extend** the **back-off time** before **retrying the call**.
- **The back-off period** should be **exponentially incremental withdrawal** to **allow sufficient correction time**.



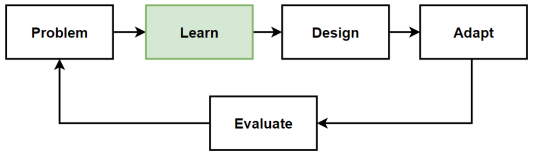
Drawbacks of Retry Pattern



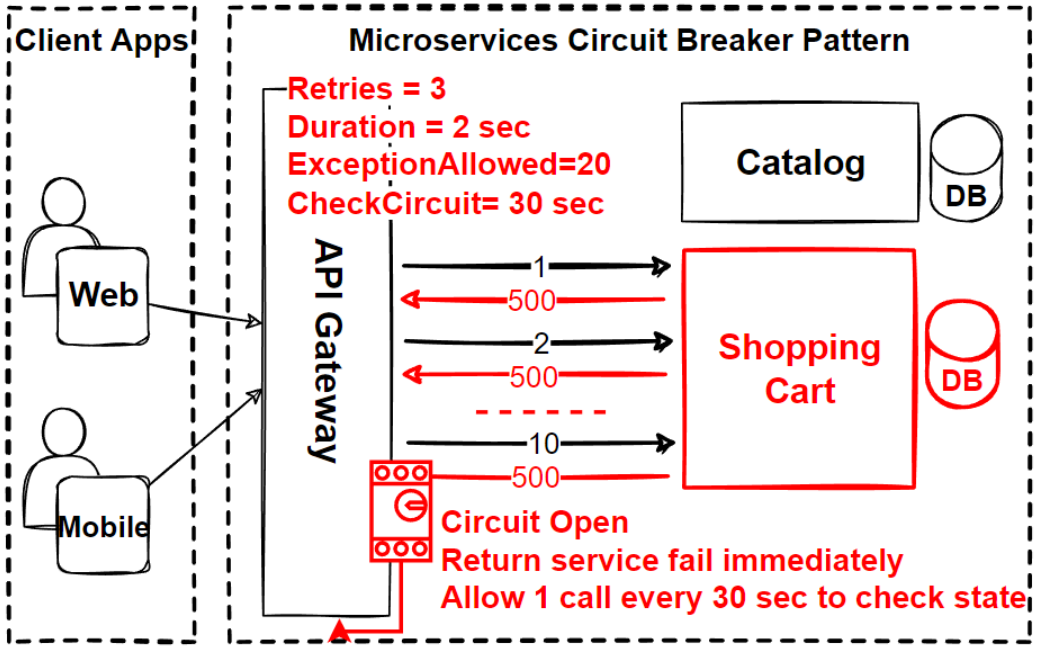
- **The Retry pattern** should be used **with caution**.
- If a service is **experiencing persistent failures** or is **unavailable for an extended period of time**, the retry pattern result in an **excessive number** of **failed requests**.
- In these cases, it is **necessary to implement additional strategies**, **circuit breaking** or **fallback** logic to **prevent the** retry pattern from **further impacting** the system.



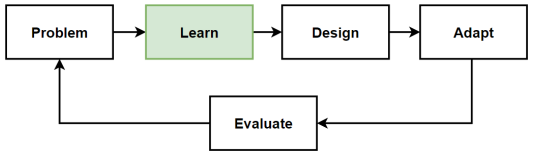
Circuit Breaker Pattern



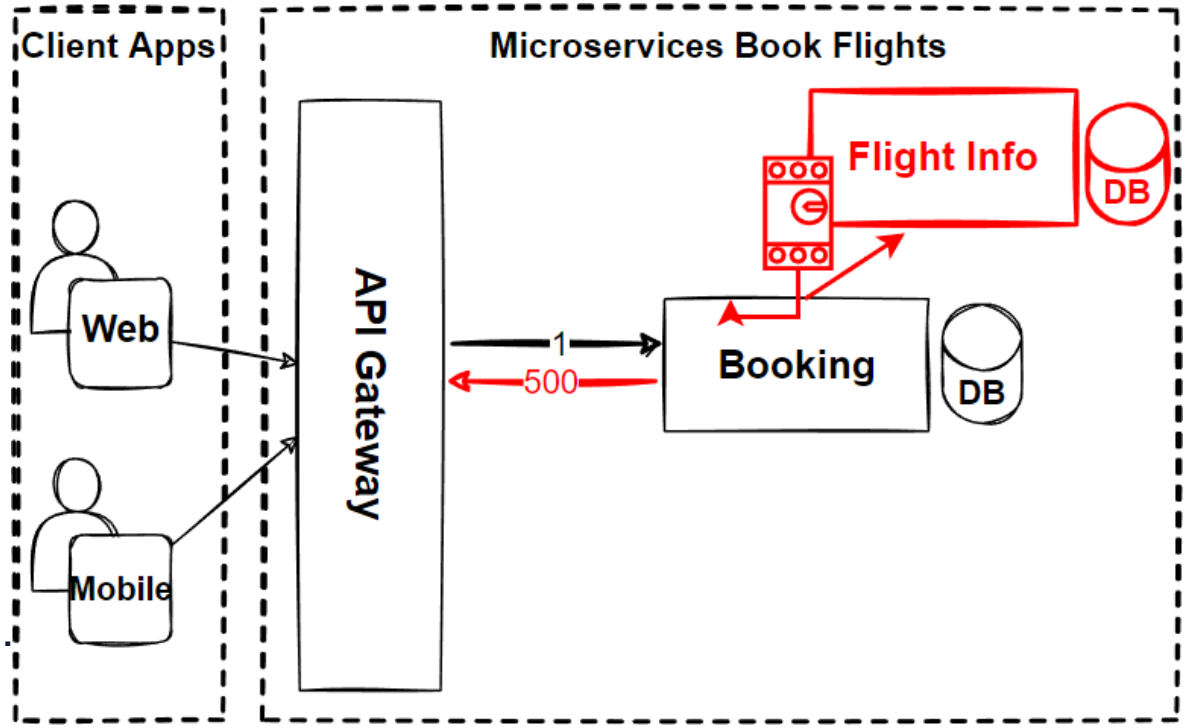
- Method in **electronic circuits** that is constructed like **circuit breaker switchgear.**
- **Stop the load transfer** in case of a failure in system to **protect the electronic circuit.**
- Protects **against failures** in **external dependencies**, useful in **microservice when failure** in **one service** can have **cascading effects** on other services.
- **Circuit breaker pattern** prevents **cascading failures** in a system.
- If one **microservice depends on another microservice** and tha second microservice fails, the **first microservice might also fail**.
- If the **first microservice** is using a **Circuit breaker pattern**, it can **prevent further calls** to the second microservice and continue to function.



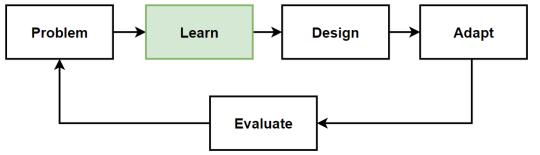
Book Flights Example



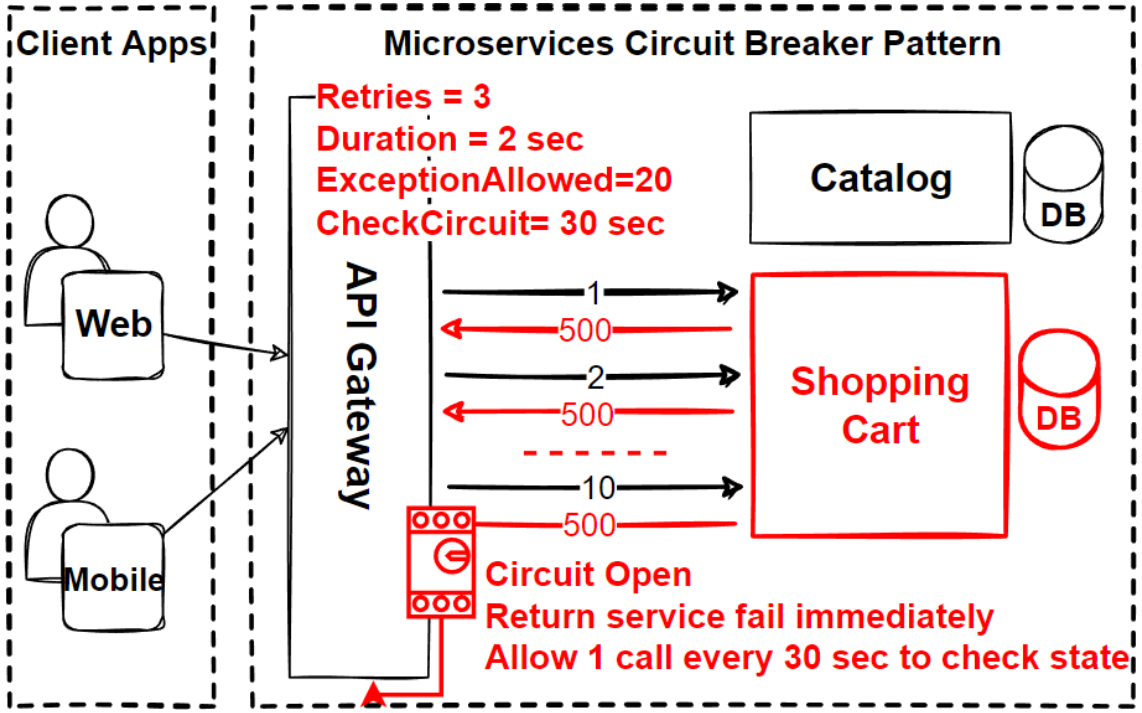
- **Microservices** that allows users to **book flights**.
- This microservice **depends on an external service** for **retrieving flight information**.
- If the **external service** becomes **unavailable**, the **flight booking** microservice will **also become unavailable**.
- To protect against this type of failure, **implement a circuit breaker pattern** around the call to the external service.
- If the **external** service **becomes unavailable**, the **circuit breaker will trip**, **preventing further calls** to the external service.
- This will help to **prevent cascading failures** in the system.



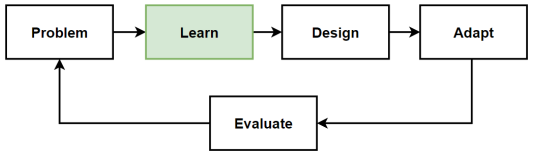
Circuit Breaker Pattern - 2



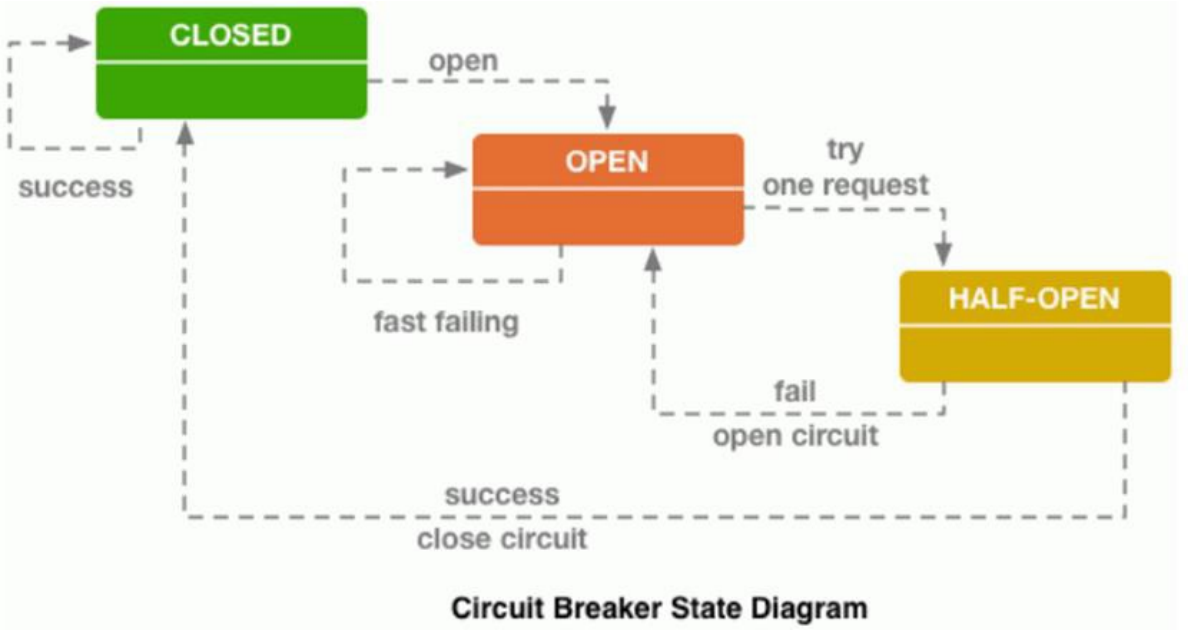
- **Circuit Breakers** pattern **monitors** the communication between the services and **follows the errors** that occur in the communication.
- When the error in the **system exceeds a certain threshold** value, **Circuit Breakers turn on** and **cut off communication**, and **returning previously** determined **error messages**.
- **While Circuit Breakers is open**, it **continues to monitor** the communication traffic.
- If the requested service **starts** to **return successful** results, it **becomes closed**.



Circuit Breaker States

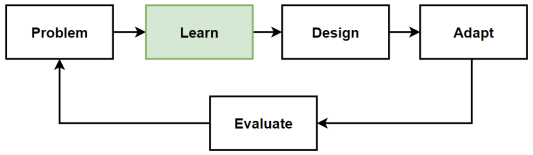


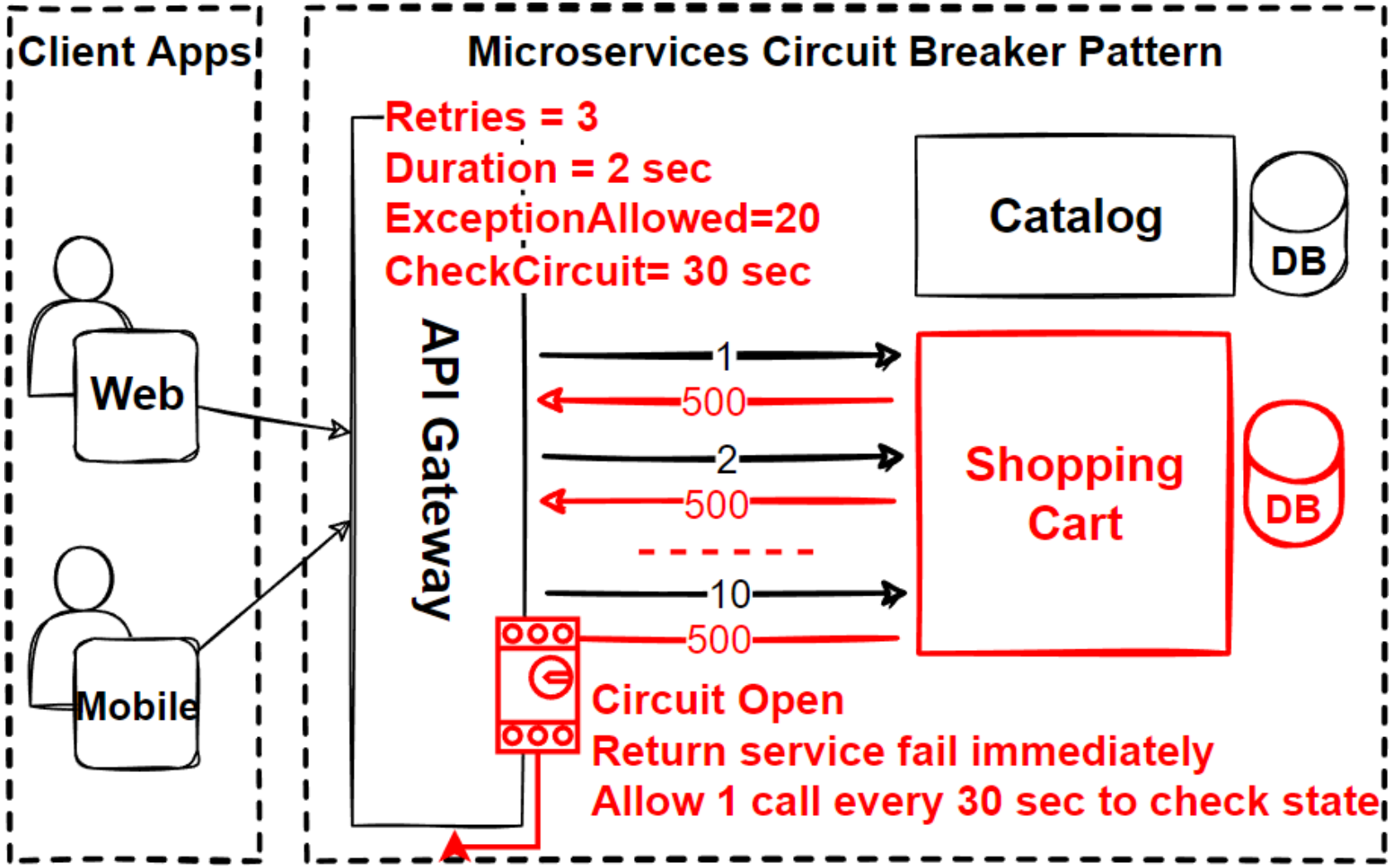
- **Closed**
  The circuit breaker is not open and all requests are executed.
- **Open**
  The Circuit Breaker is open and it prevents the application from repeatedly trying to execute an operation while an error occurs.
- **Half-Open**
  The Circuit Breaker executes a few operations to identify if an error still occurs. If errors occur, then the circuit breaker will be opened, if not it will be closed.



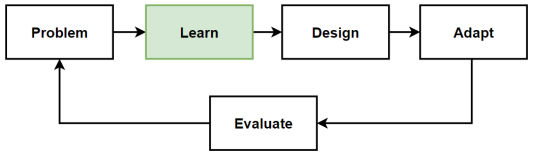
Circuit Breaker State Diagram

Circuit Breaker Pattern

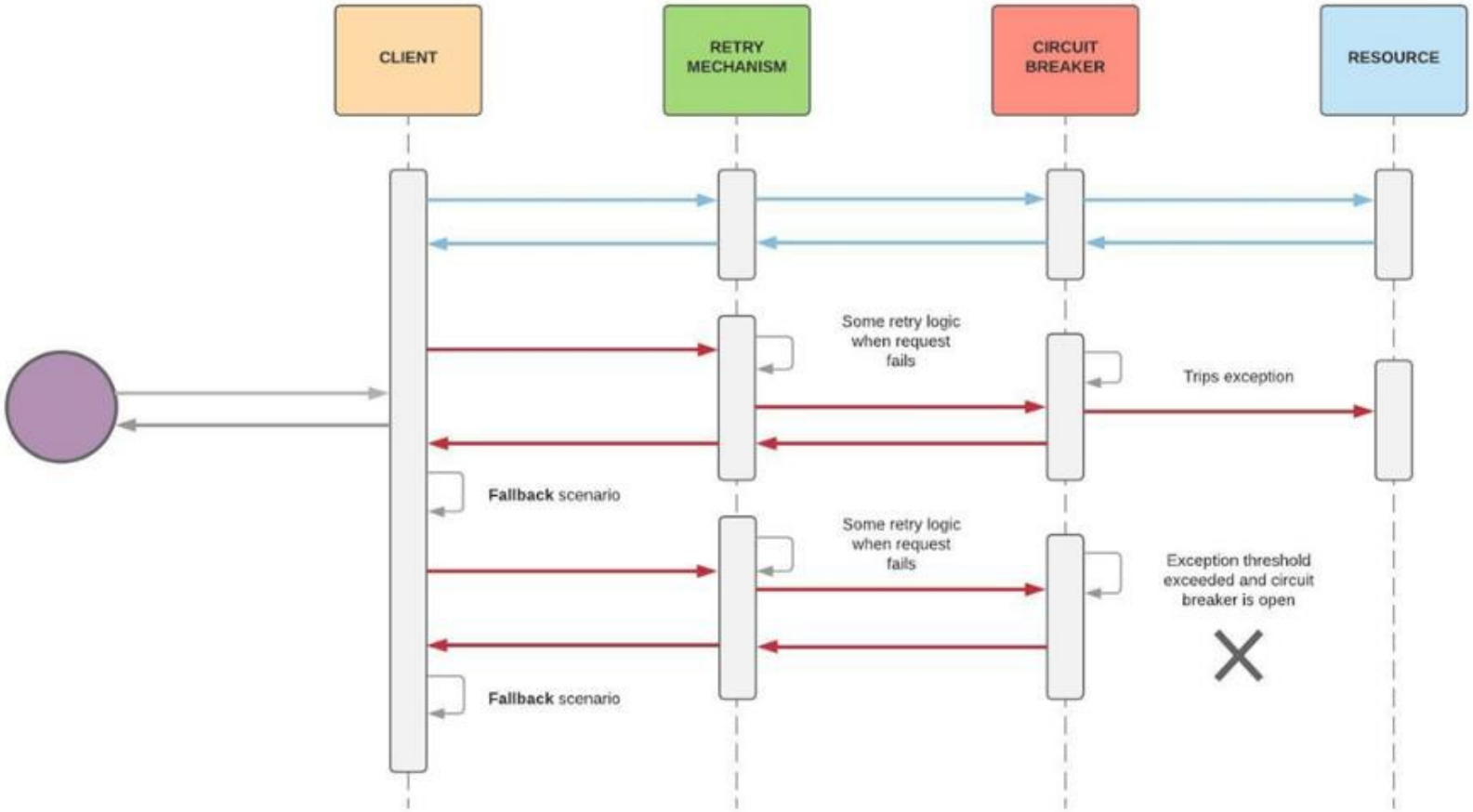




Retry + Circuit Breaker Pattern

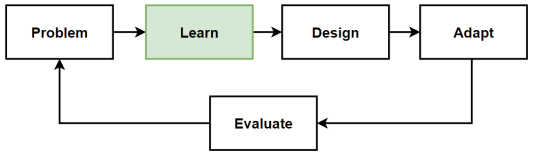


- **Retry pattern** only retry an operation in the expectation calls since it will get succeed.
- **Circuit Breaker pattern** is prevent broken communications from repeatedly trying to send request that is mostly to fail.

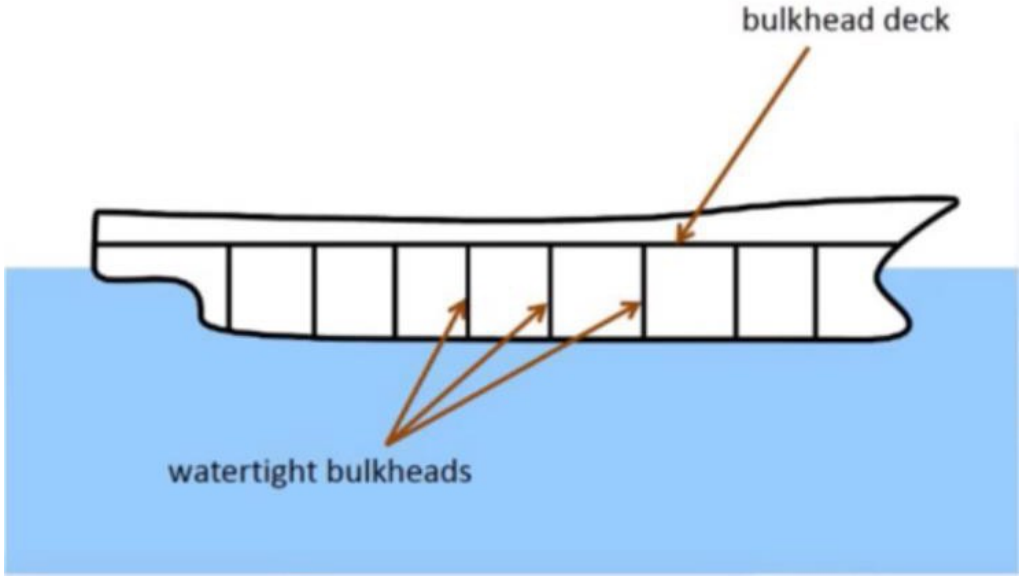


https://www.gokhan-gokalp.com/en/resiliency-patterns-in-microservice-architecture/

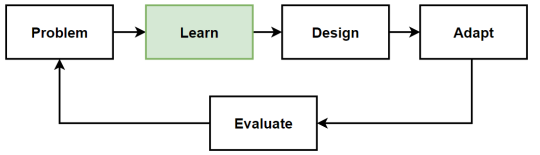
Bulkhead Pattern

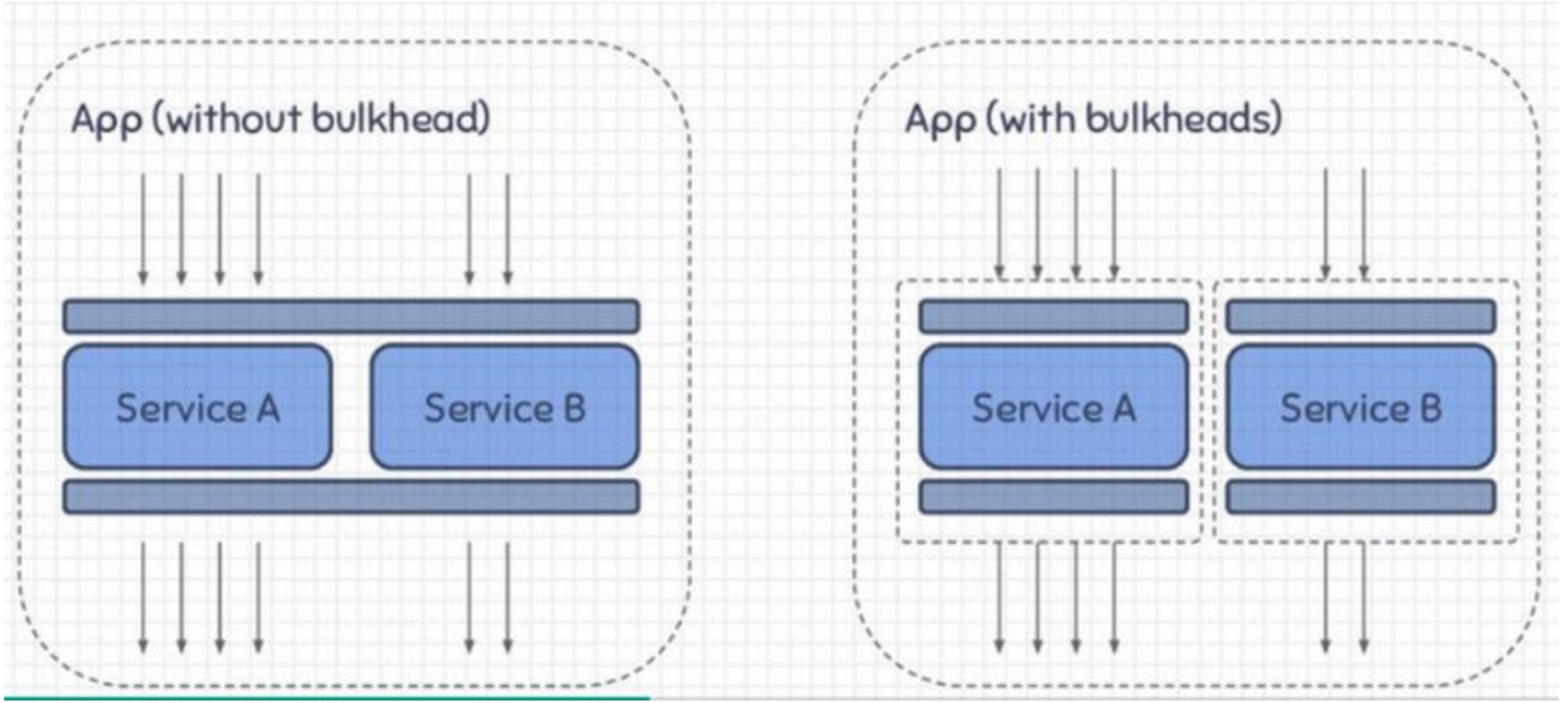


- **Bulkhead pattern** is used to **isolate failures** in a **distributed system**.
- Based on the concept of a **"bulkhead"** in a **ship**, **structural partition** that helps to **prevent the entire ship** from **flooding** if one **compartment** is **damaged**.
- In **microservices**, **isolate individual service** instances or groups of service instances from each other, to **prevent failures** in one part of the system from **affecting the entire system**.
- **One location does not affect** other services by **isolating** the services. The main purpose is **isolated error place** and **secure** the **rest** of **services**.
- **Naval architecture;** ships or submarines are made not as a whole, but by shielding from certain areas, if **there is happens** a **flood or fire**, the **relevant compartment** is **closed** and **isolated**.
- **Microservices** able to respond to user requests by **isolating** the **relevant service** so the error.



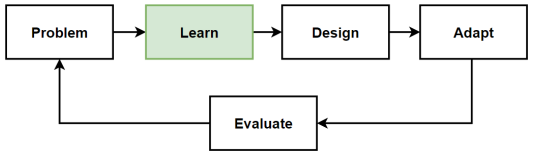
Bulkhead Pattern - 2



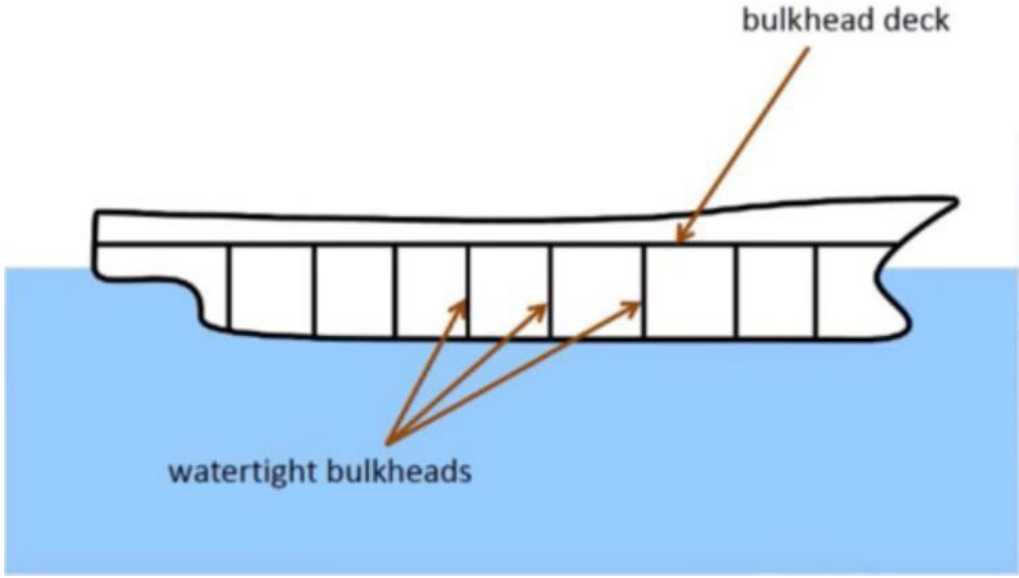


https://speakerdeck.com/slok/resilience-patterns-server-edition?slide=33

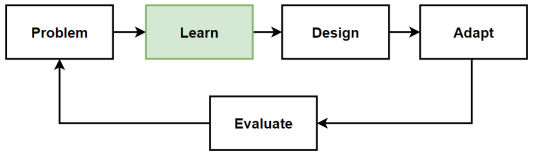
When to use Bulkhead Pattern



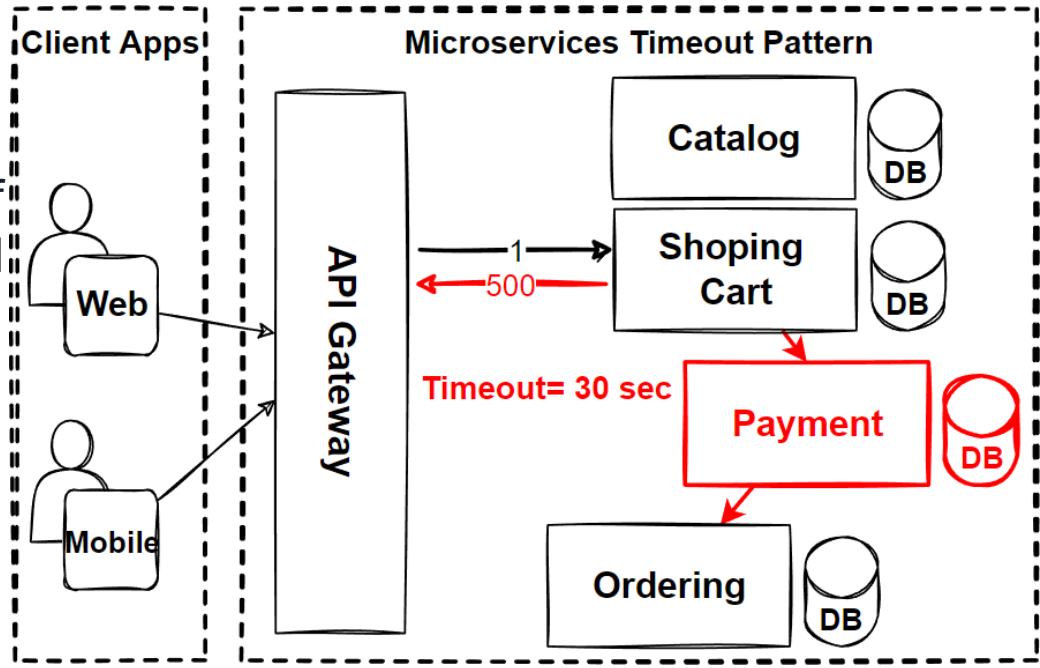
- **Microservices** use **Bulkhead Pattern** to **prevent failures** in one part of the system from affecting the entire system.
- **Use cases for bulkhead pattern:**
- When a **service has a high load** and there is a **risk of resource contention** with other service instances.
- When a **service depends on one or more downstream** services, and there is a **risk of cascading failures.**
- If a downstream **service becomes unavailable** or experiences a performance issue.
- When a service is **required to make frequent calls** to a downstream service, and there is a **risk of overloading** the downstream service if it **becomes unavailable**.



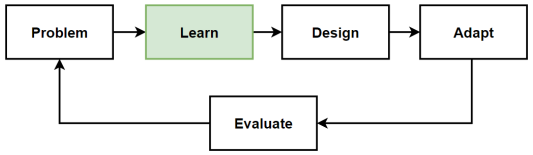
Timeout Pattern



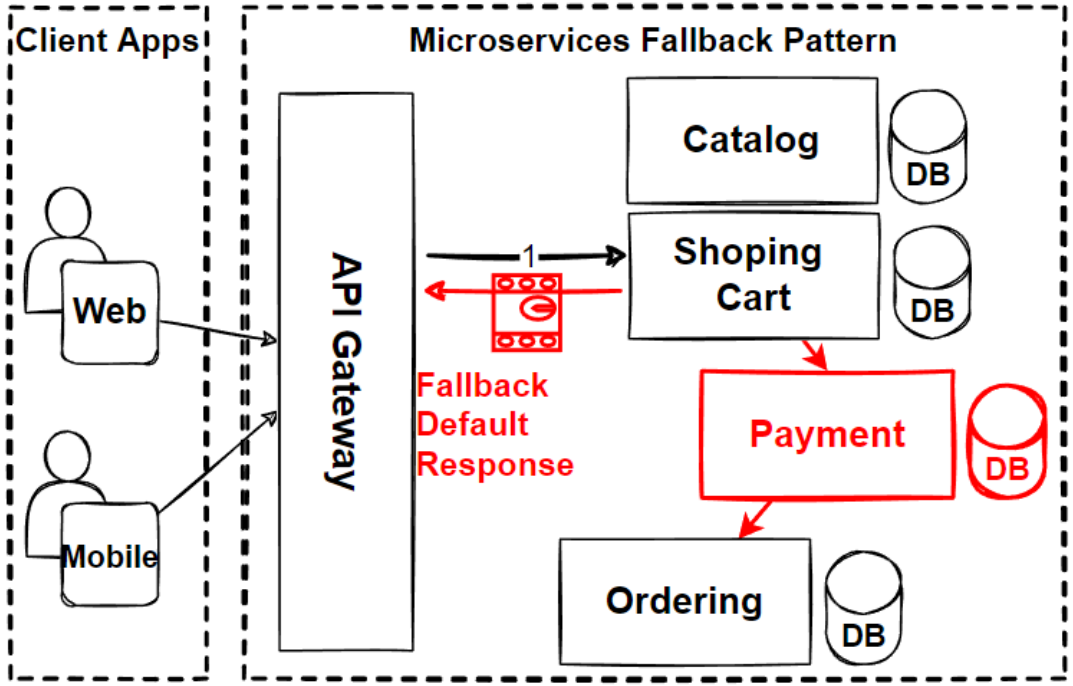
- **Timeout Pattern** provides that **should not wait for a service response** for an **indefinite amount of time**, throw an exception instead of **waiting too long**.
- It is used to handle scenarios where a **service call takes longer** than **expected to complete**.
- Setting a **maximum time limit** for the service **call to complete**, if the **time limit is exceeded**, the call is considered to have **«timed out.»**
- In microservices, It used to **prevent service calls** from **taking too long to complete.**
- If the **payment** processing service **takes longer than expected** to complete a request, the **timeout pattern** used to **cancel the request** and **return an error** to the shopping cart service.



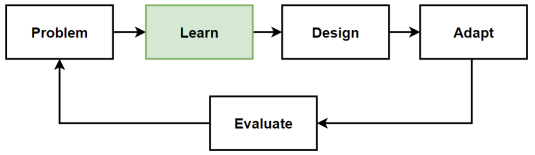
Fallback Pattern



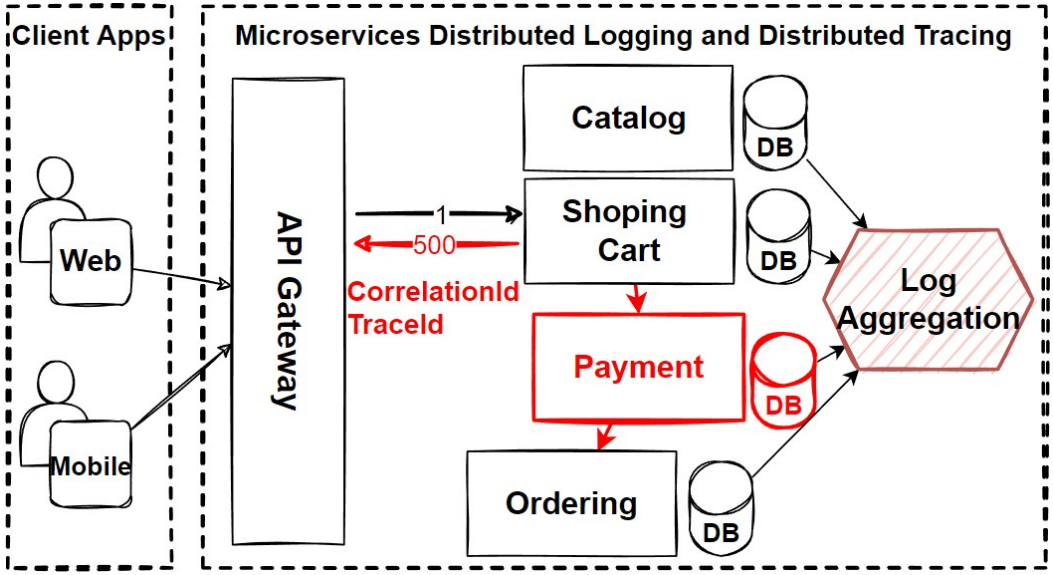
- **Fallback Pattern** provides an **alternative behavior** or response if a **request fails** or **times out**.
- If a **service is unavailable**, the client could **use a cached version** of the **data or display a default message** to the user.
- It is used to provide an **alternative course of action** when a **service call fails** or **takes too long to complete**.
- Define a **fallback function** that is called if the **service call fails** or **times out**, and the function provides an **alternative response**.
- In **microservices**, the **fallback pattern** help to **improve** the **overall reliability** and **stability of the system**.
- If the **payment** processing **service is unavailable** or takes too long to complete a request, the **fallback pattern** could be used to **provide an alternative response**.
- I.e. **displaying an error message** or offering the user the option to try again later.



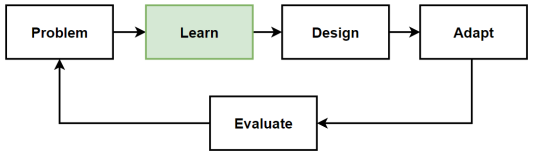
Microservices Observability with Distributed Logging and Distributed Tracing



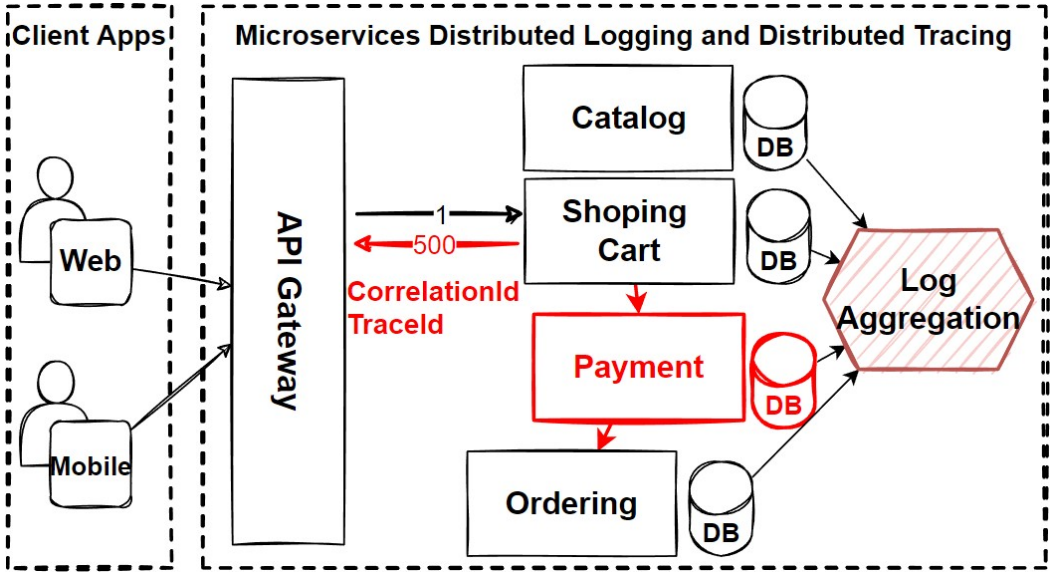
- **Microservice** have a **strategy** for **monitoring** and **managing** the **complex dependencies** on microservices
- Need to implement **microservices observability** with using **distributed logging** and **tracing features**.
- **Microservices Observability** gives us greater **operational insight.**
- **Monitor** and understand the **behavior** and **performance** of a **system** made up of **microservices**.
- **Distributed Logging** and **Distributed Tracing** are two key tools that improve observability in microservices.
- **Distributed Logging** is a practice of **collecting**, **storing**, and **analyzing log** data **from multiple service** instances.
- **Behavior of the system** over time, **identifying patterns** and **trends**, and **troubleshooting issues**.



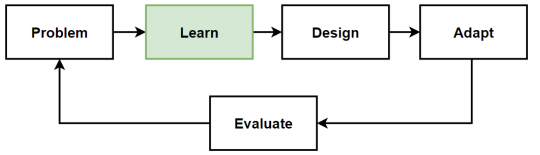
Microservices Observability with Distributed Logging and Distributed Tracing



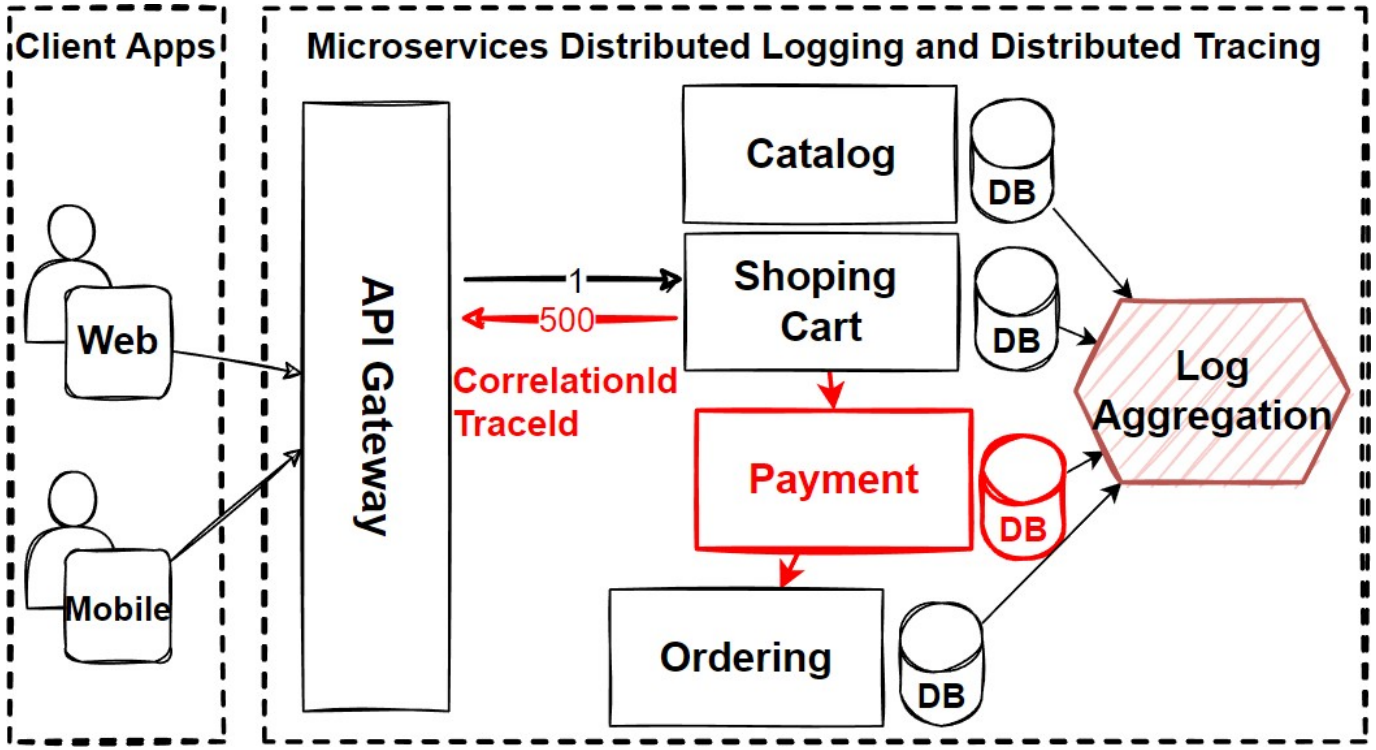
- **Distributed Tracing** is **tracking the flow of requests** through a microservices architecture, to see **how the different service** instances **interact with each other**.
- See the **performance of the system**, **identifying bottlenecks**, and **troubleshooting issues**.



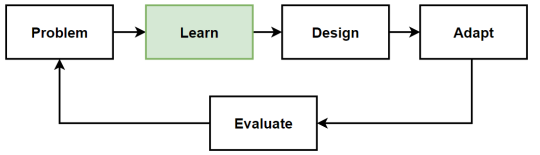
Real-world Example of Microservices Observability with Distributed Logging and Distributed Tracing



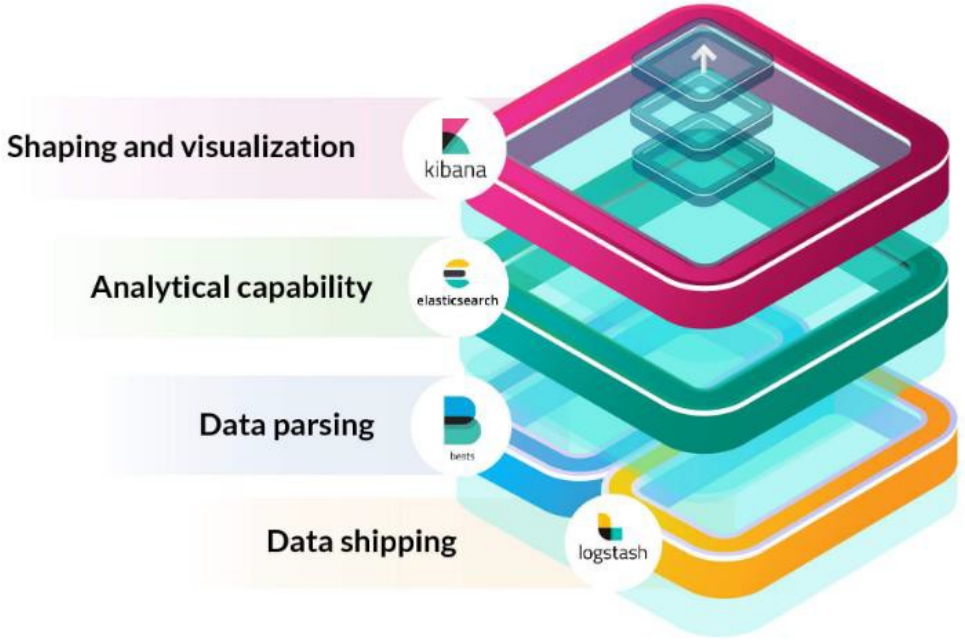
- Implement **distributed logging** to **collect**, **store**, and **analyze log data** from service instances, include information about **service calls**, **errors**, **performance metrics**, and other **relevant data**.
- Implement **distributed tracing** to **track the flow of requests** through the system.
- If a user **adds an item to their shopping cart**, the **trace with information** about the request flows from the **shopping cart** service to the **payment** processing service to the **product catalog** service.



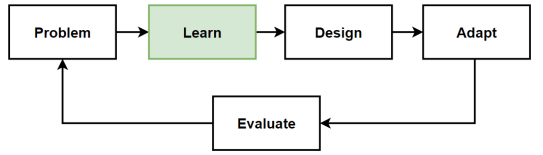
Elastic Stack for Microservices Observability with Distributed Logging



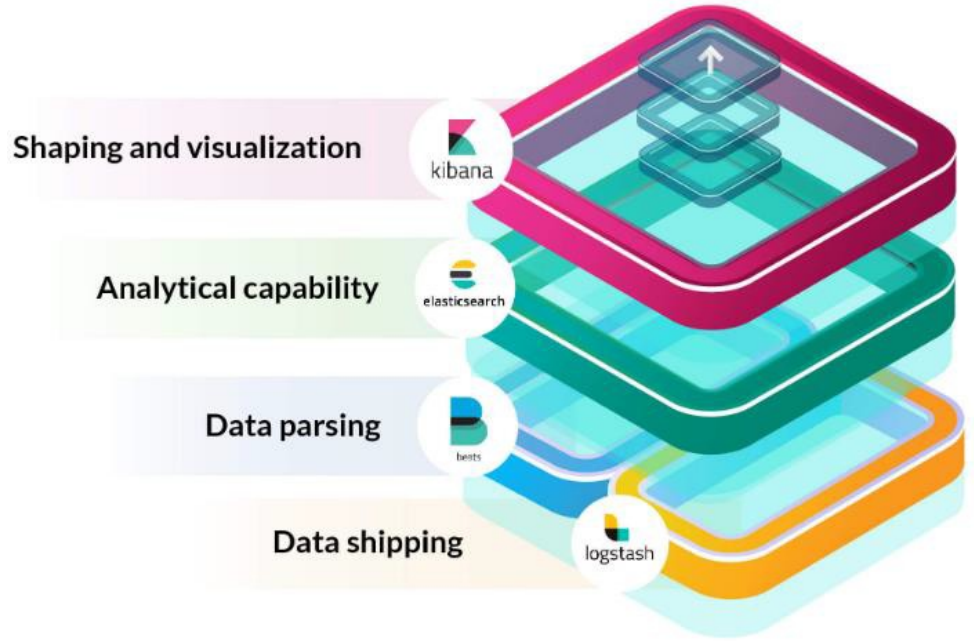
- **Elastic Stack** is a collection of **open-source tools** for **collecting**, **storing**, and **analyzing log data** and other types of data.
- Used for **microservices observability** to **provides** a **flexible** and **scalable** platform for **monitoring** and understanding the behavior.
- **Elasticsearch**
  Distributed search and analytics engine that can be used to store and index log data and other types of data.
- **Logstash**
  Data collection and transformation tool that used to collect log data from different sources and send it to Elasticsearch.
- **Kibana**
  Data visualization and exploration tool that used to create dashboards and visualizations based on data.
- **Beats**
  Collection of lightweight data shippers that used to collect log data and send it to Elasticsearch.



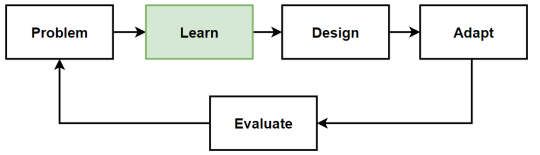
Why logging with ElasticSearch and Kibana in Microservices ?

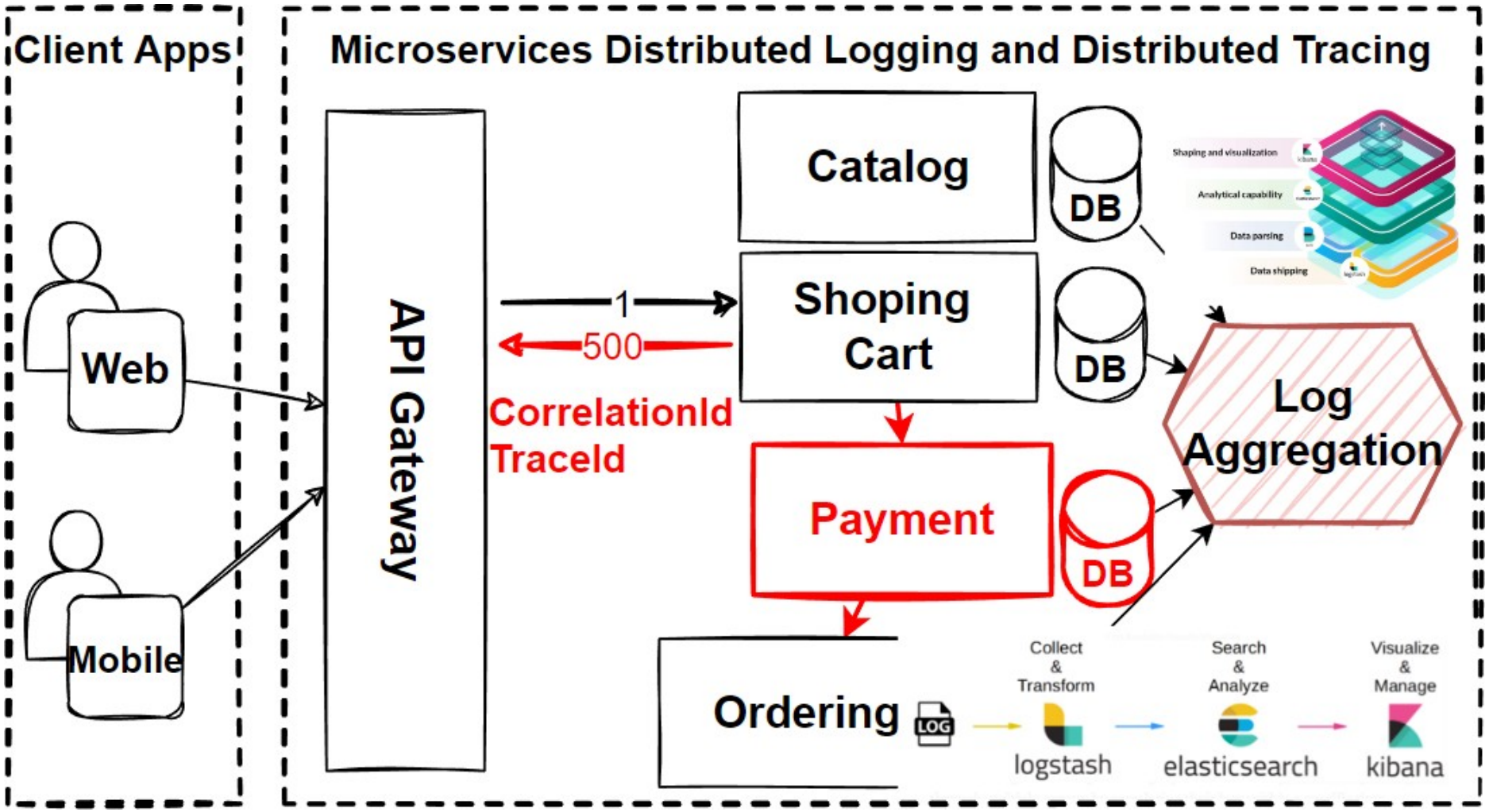


- **Logging** is **critical** to **implement** to **identify problems** in distributed architecture.
- **Elastic Stack** used to **collect**, **store**, and **analyze log** data from multiple service instances in a microservices architecture.
- Useful for **understanding** the **behavior** of the system over time, **identifying patterns** and **trends**, and **troubleshooting issues**.
- **Elastic Stack** might be used to **collect log** data from each service instance.
- **Kibana** to **visualize** and **analyze** the data in order to **identify patterns** or **trends**.

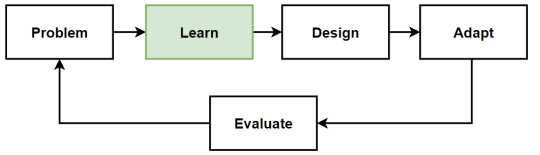


Real-world Example with ElasticStack

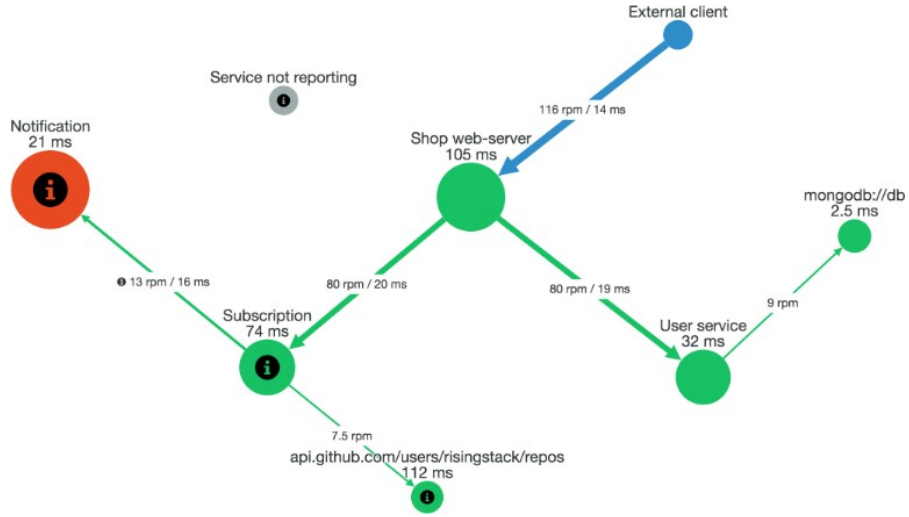


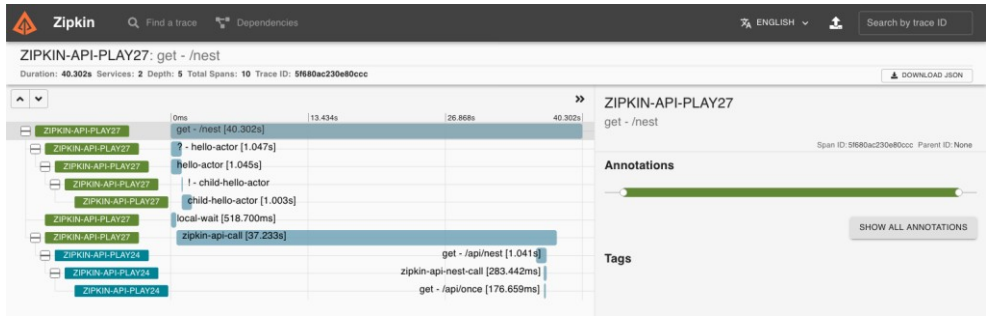


Distributed Tracing with OpenTelemetry using Zipkin

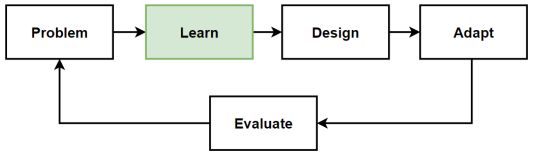


- **Distributed Tracing** is used to **track the flow of a request** as it is processed by different microservices in a system.
- **Different microservices** are interacting and **identify issues** or **bottlenecks** and **troubleshooting issues** in the system.
- **OpenTelemetry** is an **open-source project** that provides a set of APIs for **collecting** and **exporting telemetry data**; **traces**, **metrics**, and **logs**.
- **Zipkin** is a **distributed tracing system** that **collects** and **stores trace** data from microservices, provides a web UI for **viewing** and **analyzing trace data**.
- To use **OpenTelemetry with Zipkin** for **microservices distributed tracing**, **OpenTelemetry SDK** would be **integrated.**
- **Allows to collect trace data** as requests flow through the system, and **send data** to a **Zipkin server** for **storage** and **visualization**.

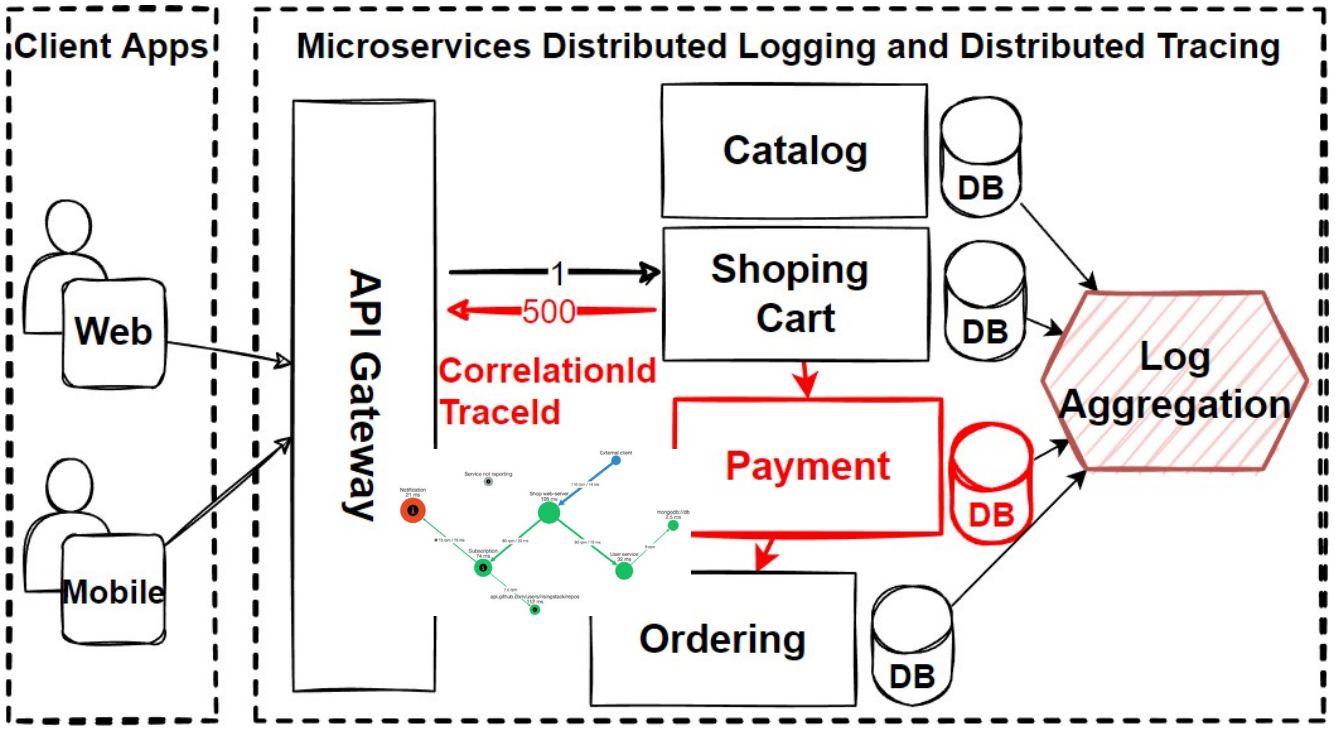


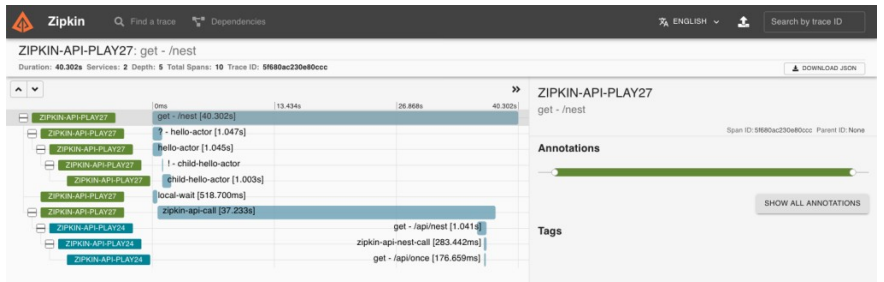


Real-world Example of Distributed Tracing with OpenTelemetry using Zipkin

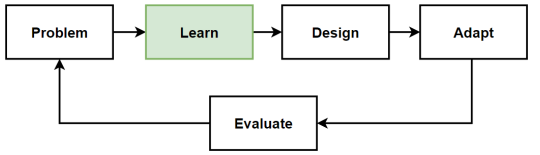


- Use **OpenTelemetry** with **Zipkin** to **collect trace** data **as requests flow** through the system.
- When a **user adds** an item to their **shopping cart**, the **trace include information about** the **request**.
- The **trace data collected by** the **OpenTelemetry SDKs** integrated into each service instance, and then **sent** to a **Zipkin server** for **storage** and **visualization**.
- Use the **Zipkin UI** to **visualize** and **analyze** the **trace data**, to **identify patterns** or **trends**.

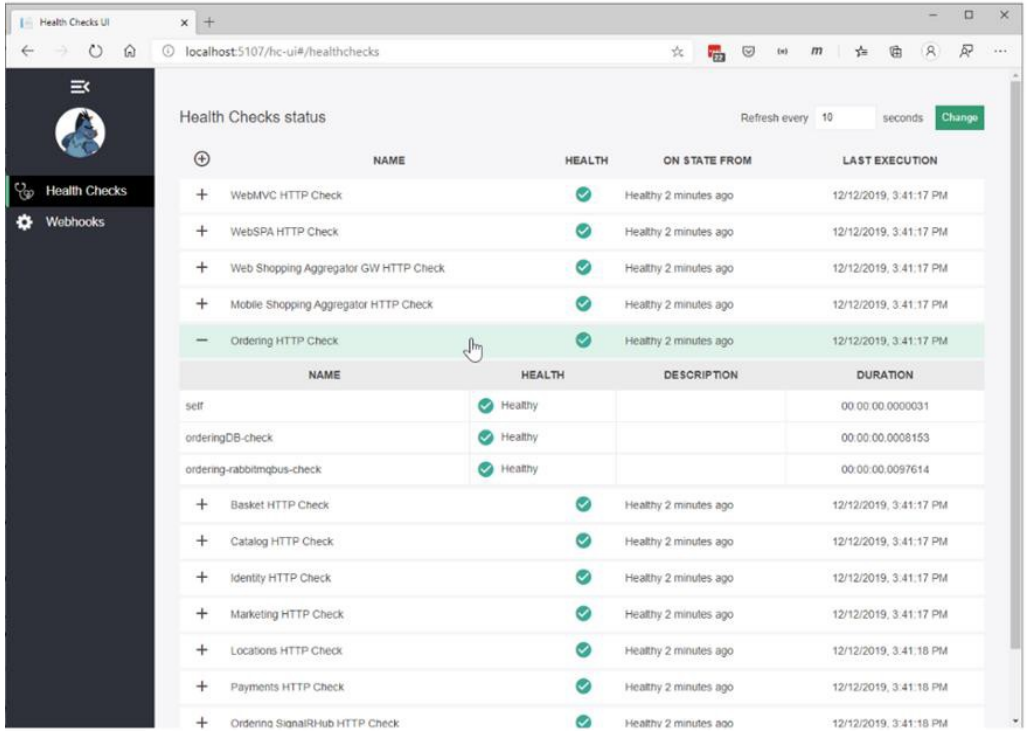




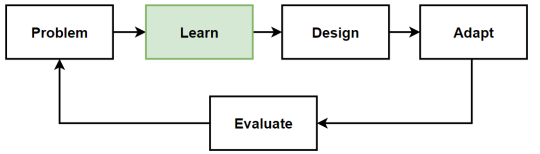
Microservices Health Checks: Liveness, Readiness and Performance Checks



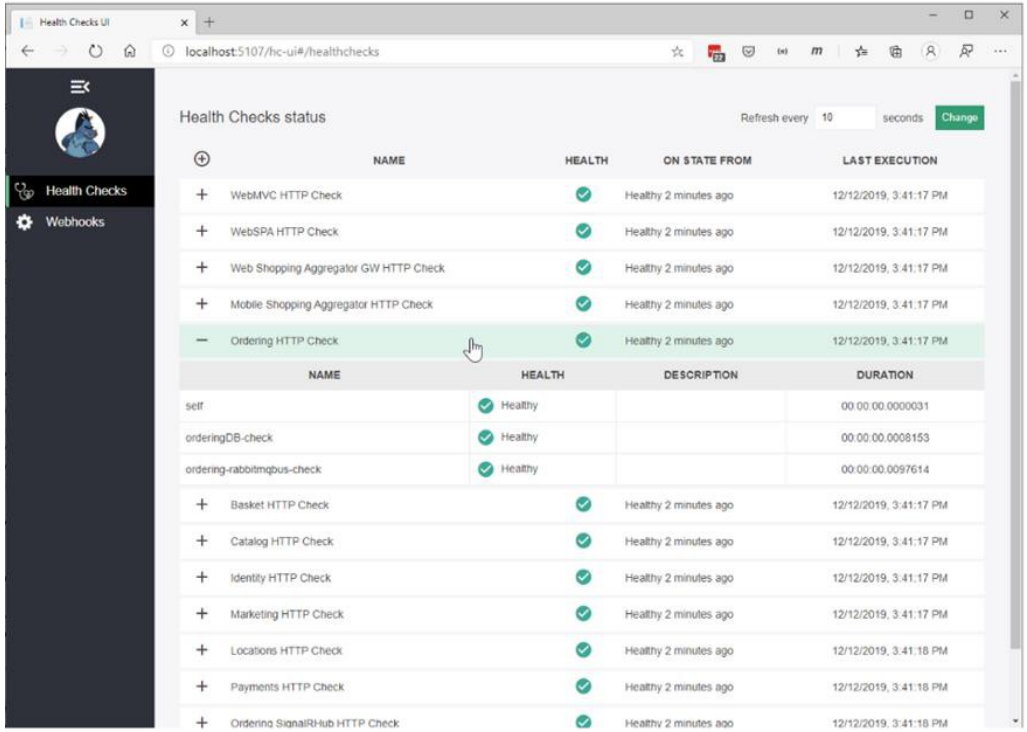
- The **process of monitoring** the **health** and **performance** of **individual microservices** in a system.
- The **failure of a single microservice** can have **cascading effects** on the rest of the system, **important to identify** and **address issues**.
- What is **Health Checks** for **microservices** ?
- **Health Checks for microservices** are a way to **monitor the health** and **performance** of **individual microservices** in a system.
- **Health checks** used to **determine whether** a **microservice** is **functioning properly** and is **able to handle requests**.
- There are **3 types of health checks** that can be used for microservices.



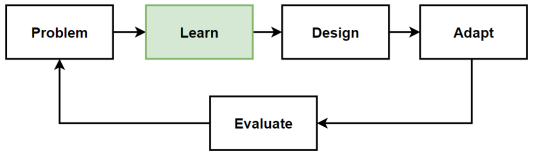
Microservices Health Checks: Liveness, Readiness and Performance Checks



- **Liveness Checks**
  Determine whether a microservice is still running. If a liveness check fails, it may indicate that the microservice has crashed.
- **Readiness Checks**
  Determine whether a microservice is ready to handle requests. If a readiness check fails, it may indicate that the microservice is not yet ready to handle traffic.
- **Performance Checks**
  Monitor the performance of a microservice, such as response times or error rates. If the results of a performance check indicate that a microservice is not performing as expected.



Microservices Health Monitoring with Kubernetes, Prometheus and Grafana



- **Use Liveness and Readiness Probes**
  Kubernetes provides liveness and readiness probes that can be used to monitor the health of individual microservices.
- Liveness probes check to see if a microservice is still running, and readiness probes check to see if a microservice is ready to receive traffic.
- **Use Monitoring Tools**
  Monitoring tools can be used to monitor the health and performance of microservices that can be integrated with Kubernetes to provide alerts or notifications when issues arise. I.e. Prometheus, Grafana, Datadog.
- **Use Log Analysis Tools**
  Analyze log messages generated by your microservices and identify issues or trends. Elastic Stack (Elasticsearch, Logstash, Kibana), Fluentd, Splunk.
- **Set up Alerts and Notifications**
  Setting up alerts and notifications can help to ensure that relevant parties are notified when issues arise, allowing them to be addressed quickly. Slack, Teams, Email, SMS.

Before Design – What we have in our design toolbox ? - Old

Architectures

- **Microservices Architecture**
- Event-Driven Microservices Architecture

Patterns&Principles

- The Database-per-Service Pattern, Polygot Persistence, Decompose services by scalability, The Scale Cube
- Microservices Decomposition Pattern
- Microservices Communications Patterns
- Microservices Data Management Patterns
- Event-Driven Architecture
- Microservices Distributed Caching
- **Microservices Deployments with Containers and Orchestrators**

Microservices Deployment

- **Docker and Kubernetes Architecture, Helm Charts**
- **Kubernetes Patterns; Sidecar Patterns, Service Mesh Pattern**
- **DevOps and CI/CD Pipelines**
- **Deployment Strategies; Blue-green, Rolling, Canary and A/B Deployment.**
- **Infrastructure as code (IaC)**

Non-FR

- High Scalability
- High Availability
- Millions of Concurrent User
- Independent Deployable
- Technology agnostic
- Data isolation
- Resilience and Fault isolation

FR

- List products
- Filter products as per brand and categories
- Put products into the shopping cart
- Apply coupon for discounts
- Checkout the shopping cart and create an order
- List my old orders and order items history

Before Design – What we have in our design toolbox ? - New

Architectures

- **Microservices Architecture**
- Event-Driven Microservices Architecture

Patterns&Principles

- The Database-per-Service Pattern, Polygot Persistence, Decompose services by scalability, The Scale Cube
- Microservices Decomposition Pattern
- Microservices Communications Patterns
- Microservices Data Management Patterns
- Event-Driven Architecture
- Microservices Distributed Caching
- Microservices Deployments with Containers and Orchestrators

Microservices Resilience

- **Resilience Patterns; Retry, Circuit-Breaker, Bulkhead, Timeout, Fallback Pattern**
- **Distributed Logging and Distributed Tracing**
- **Elastic Stack; Elasticsearch + Logstash + Kibana**
- **OpenTelemetry using Zipkin**
- **Kubernetes Health Monitoring with tools like Prometheus and Grafana**

Non-FR

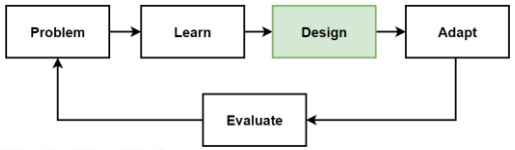
- High Scalability
- High Availability
- Millions of Concurrent User
- Independent Deployable
- Technology agnostic
- Data isolation
- Resilience and Fault isolation

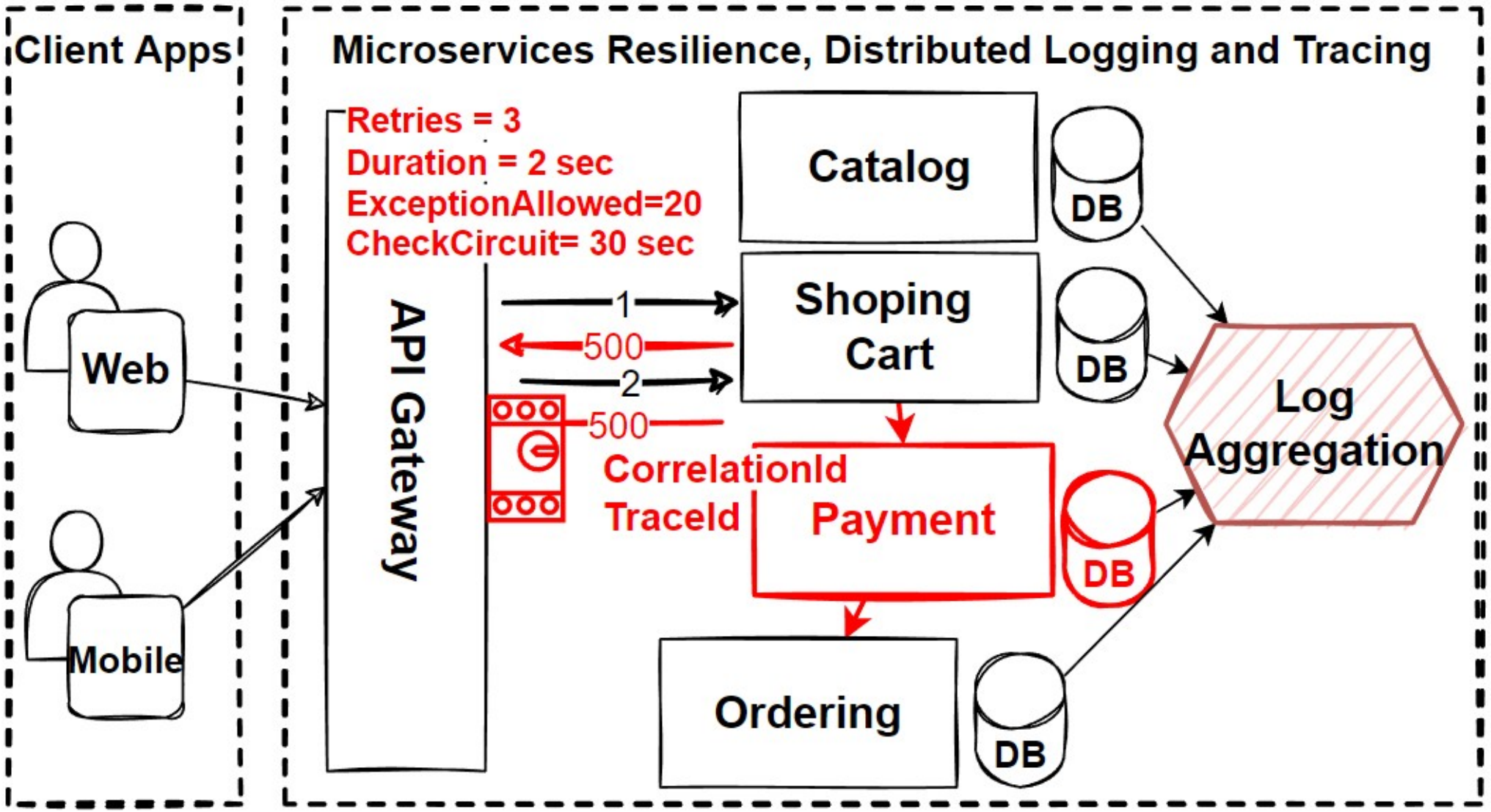
FR

- List products
- Filter products as per brand and categories
- Put products into the shopping cart
- Apply coupon for discounts
- Checkout the shopping cart and create an order
- List my old orders and order items history

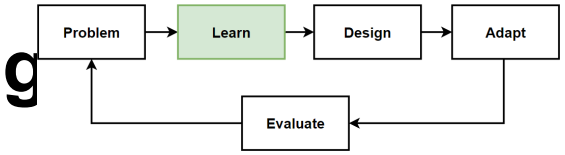
- **Microservices Resilience, Observability and Monitoring**

Microservices Resilience, Observability and Monitoring



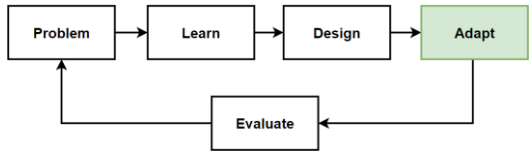


Microservices Resilience, Observability and Monitoring



- **Improved fault tolerance**
  Help a microservices system continue functioning even in the face of failures or other disruptions, making it more fault tolerant.
- **Improved uptime and availability**
  Implementing patterns circuit breakers and fallback mechanisms, you can ensure that your system remains available even when individual components get problems.
- **Increased reliability**
  When a microservices system is designed to be resilient, it is less likely to experience failures or disruptions.
- **Enhanced scalability**
  Microservices architecture is designed to be scalable, and using resilience patterns can further enhance this capability.
- The bulkhead pattern can allow you to scale different components of your system independently, without affecting the others.

Adapt: Microservices Resilience, Observability and Monitoring



Frontend SPAs

- Angular
- Vue
- React

API Gateways

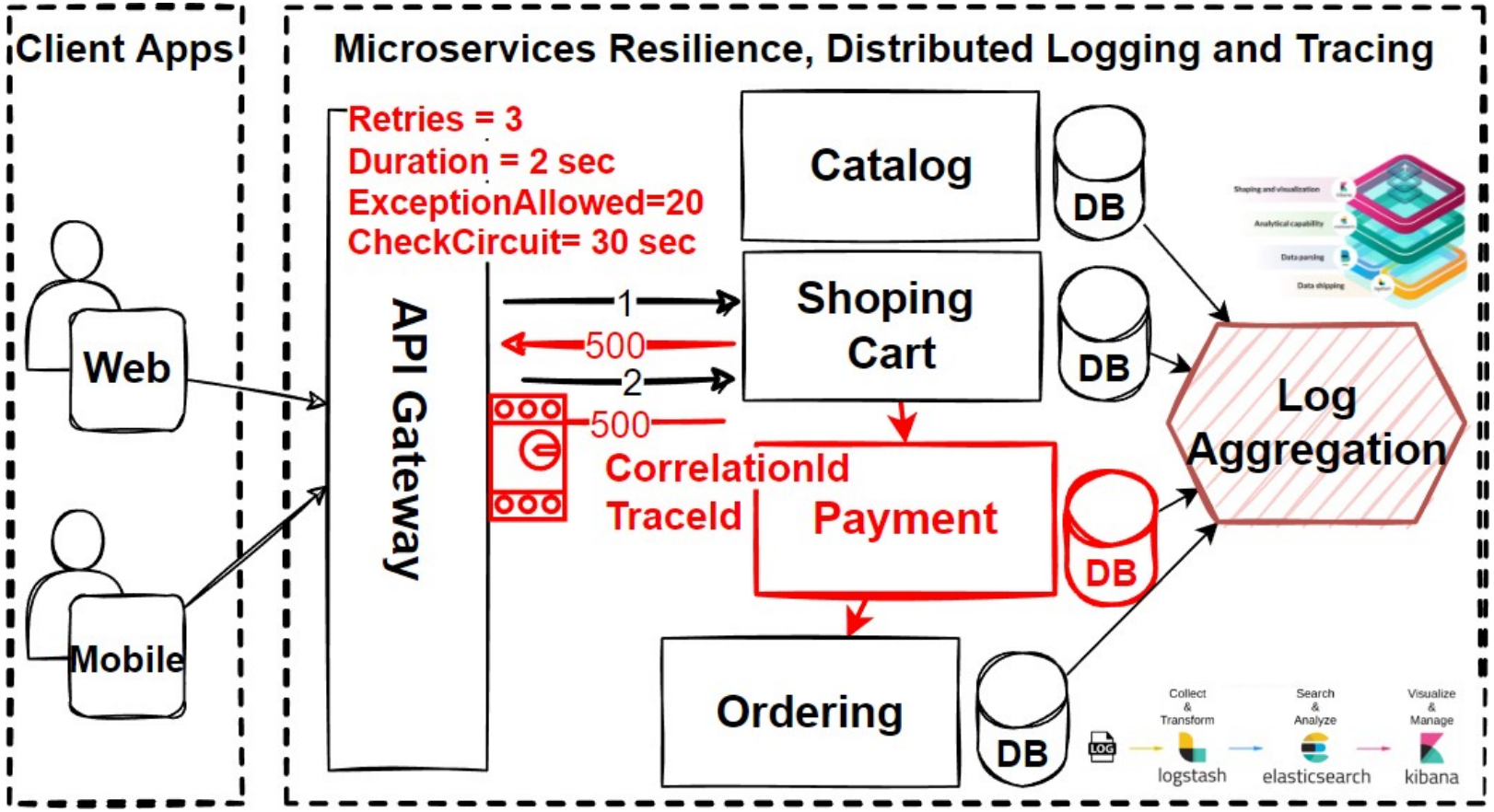
- Kong Gateway
- Express Gateway
- Amazon AWS API Gateway

Backend Microservices

- Java – Spring Boot
- .Net – Asp.net
- JS – NodeJS

Resiliency Pattern

- Retry
- Circuit breaker
- Bulkhead
- Timeout, Fallback



Distributed Logging

- Elastic Stack
- Fluentd
- Splunk

Cloud Distributed Logging

- AWS Elastic Cloud
- Azure Elastic

Distributed Tracing

- OpenTelemetry
- Zipkin
- AWS X-Ray

Monitoring

- Kubernetes Health Checks
- Prometheus – Grafana, Datadog

Cloud Monitoring

- Azure Application Insights
- Amazon CloudWatch

CÁM ƠN ĐÃ CHÚ Ý LẮNG NGHE!